Faculty of Engineering

Department of Informatics Engineering
Software Engineering and Information Systems

كلية الهندسة

قسم الهندسة المعلوماتية

هندسة البرمجيات ونظم المعلومات

نظام إدارة مركز صحي

Health center management system

Senior-2 Project- Completed the requirements for obtaining a bachelor's degree in Software Engineering and Intelligent Systems

Prepared By:

Roaa Kashour

Tasneem Kadoor

Supervised By

Dr. Akram Masouh

October 2026

SUPERVISOR CERTIFICATION

I Certify that the Preparation of this Project entitled by ……, prepared by ……..…

……

……., was made under my supervision at Department of Informatics Engineering in partial fulfillment of the Requirements for the Degree of Bachelors of Software Engineering and Information Systems

Name: …… Signature: …..…….. Date:

ملخص البحث

تعاني المراكز الصـــحية للرعاية الأولية في كثير من الأحيان من تجزؤ العمليات اليدوية والورقية، مما يؤدي إلى تكرار البيانات، وعدم كفاءة الجدولة، وضعف التفاعل مع المرضى. لمعالجة هذه التحديات، يهدف هذا المشـروع إلى تصـميم وتطوير "نظام إدارة مركز صـحي" (HCMS) شـامل يعتمد على الويب ومركزي، بهدف أتمتة العمليات الإدارية والسريرية والمالية. يعتمد المشروع على منهجية التطوير التكراري والتزايدي (Iterative and Incremental) لتســليم وحدات النظام والتحقق منها تدريجياً. يوفر الحل المقترح منصـــة موحدة وآمنة تدمج خمســـة أدوار مســـتخدمين متميزة: المدير، الطبيب، الأخصـــائي، الموظف، والمريض. تم بناء النظام باسـتخدام حزمة تقنيات قوية تشـمل PHP لمنطق الخلفيـة، وMySQL لإدارة قواعـد البيـانـات العلائقيـة، وHTML5/CSS3/JavaScript لواجهـة أمامية متجاوبة وتدعم اللغة العربية (RTL). يتم ضمان الأمان وسلامة البيانات من خلال التحكم في الوصـــول القائم على الأدوار (RBAC)، واســتخدام PDO لمنع حقن SQL. أكدت مراحل الاختبار والتقييم استقرار النظام، حيث حقق نسبة نجاح 100% عبر 55 حالة اختبار، غطت جميع المتطلبات الوظيفية الـــ 54. أظهرت مقاييس الأداء أوقات تحميل متوسطها أقل من 1.5 ثانية. في النهاية، نجح النظام في القضـاء على عزلة البيانات، وتسريع العمليات السريرية، وتعزيز تفاعل المرضـى، مما يوفر حلاً رقمياً فعالاً من حيث التكلفة وقابلاً للتوسع للمراكز الصحية متوسطة الحجم.

الكلمات المفتاحية: نظام إدارة المركز الصـحي، تطبيق ويب، التحكم في الوصـول القائم على الأدوار، التطوير التكراري، المعلوماتية الصحية، PHP/MySQL.

Abstract

Primary healthcare centers frequently struggle with fragmented, manual workflows that lead to data redundancy, scheduling inefficiencies, and limited patient engagement. To address these challenges, this project aims to design and develop a comprehensive, web-based Health Center Management System (HCMS) that centralizes administrative, clinical, and financial operations. The project adopts an Iterative and Incremental Development methodology to progressively deliver and validate system modules. The proposed solution integrates five distinct user roles—Administrator, Doctor, Specialist, Staff, and Patient—into a unified, secure platform. The system was built using a robust technology stack comprising PHP for backend logic, MySQL for relational database management, and HTML5/CSS3/JavaScript for a responsive, Arabic-localized (RTL) frontend. Security and data integrity are ensured through Role-Based Access Control (RBAC), PDO prepared statements, and secure session management. Testing and evaluation phases confirmed the system's stability, achieving a 100% pass rate across 55 test cases, covering all 54 functional requirements. Performance metrics demonstrated average page load times under 1.5 seconds and high usability scores. Ultimately, the HCMS successfully eliminates data silos, accelerates clinical workflows (such as laboratory test requests and medical record management), and fosters patient engagement through integrated feedback and health education modules, providing a cost-effective, scalable digital transformation solution for mid-sized health centers.

Keywords: Health Center Management System, Web Application, Role-Based Access Control, Iterative Development, Health Informatics, PHP/MySQL.

Index of content

CHAPTER 1 INTRODUCTION 1

1.1 Project Background: 2
1.2 Problem Statement: 2
1.3 Project Significance: 3
1.4 Project Objectives: 4
1.5 Project Scope: 5
1.6 Project Contributions: 6
1.7 Report Organization: 6

CHAPTER 2 FUNDAMENTAL CONCEPTS AND REFERENCE STUDIES 8

2.1 Fundamental Concepts: 9
2.2 Reference Studies and Previous Works: 10
2.3 Comparison of Previous Works: 11
2.4 Research Gap: 12

CHAPTER 3 PROJECT METHODOLOGY AND REQUIREMENTS ANALYSIS. 15

3.1 Adopted Development Methodology: 16
3.2 General Functional Requirements of the System: 16
3.3 Non-Functional Requirements of the System: 17
3.4 Functional Requirements of the Current Project 18
3.5 Feasibility and Risks 22
3.6 Project Plan: 24

CHAPTER 4 ANALYTICAL AND DESIGN STUDY 26

4.1 System Architecture: 27
4.2 Use Case Diagram: 29
4.2.1 Use Case Description: 31
4.3 Activity Diagram: 44
4.4 Sequence Diagram: 51
4.5 Class Diagram: 60
4.6 Component Diagram: 61
4.7 Entity-Relationship Diagram (ERD): 62
4.8 System Sitemap: 63
4.9 Initial Test List (Unit Tests): 65
4.10 AI and Algorithms Design: 66
4.11 Requirements Traceability Matrix (RTM): 66

CHAPTER 5 IMPLEMENTATION 68

5.1 Execution Environment: 69
5.2 Software Project Structure: 70

List Of Figures

FIGURE (3.1) GANTT CHART 25

FIGURE (4.1) SYSTEM BLOCK DIAGRAM 28

FIGURE (4.1) GENERAL USE CASE DIAGRAM 30

FIGURE (4.3) ACTIVITY DIAGRAM FOR DEPARTMENT MANAGEMENT 44

FIGURE (4.4) ACTIVITY DIAGRAM FOR USER MANAGEMENT 44

FIGURE (4.5) ACTIVITY DIAGRAM FOR APPOINTMENTS MANAGEMENT . 45

FIGURE (4.6) ACTIVITY DIAGRAM FOR AUTHENTICATION AND PROFILE MANAGEMENTS 45

FIGURE (4.7) ACTIVITY DIAGRAM FOR CONTENT AND SURVEYS AND FEEDBACK 46

FIGURE (4.8) ACTIVITY DIAGRAM FOR FINICAL AND REPORTING OPERATION 46

FIGURE (4.9) ACTIVITY DIAGRAM FOR VIEW DOCTORS AND SPECIALISTS LIST 47

FIGURE (4.10) ACTIVITY DIAGRAM FOR TEST REQUESTS MANAGEMENT 48

FIGURE (4.11) ACTIVITY DIAGRAM FOR MEDICAL RECORDS & DIAGNOSIS MANAGEMENT 49

FIGURE (4.12) ACTIVITY DIAGRAM FOR ARTICLES MANAGEMENT 50

FIGURE (4.13) SEQUENCE DIAGRAM FOR USERS AND DEPARTMENT MANAGEMENTS 52

FIGURE (4.14) SEQUENCE DIAGRAM FOR APPOINTMENT MANAGEMENTS 53

FIGURE (4.15) SEQUENCE DIAGRAM FOR AUTHENTICATION AND PROFILE MANAGEMENTS 54

FIGURE (4.16) SEQUENCE DIAGRAM FOR CONTENT AND SURVEYS AND FEEDBACK 55

FIGURE (4.17) SEQUENCE DIAGRAM FINICAL AND REPORTING OPERATION 56

FIGURE (4.18) SEQUENCE DIAGRAM FOR TESTS MANAGEMENT 57

FIGURE (4.19) SEQUENCE DIAGRAM FOR MEDICAL RECORDS & DIAGNOSIS MANAGEMENT 58

FIGURE (4.20) SEQUENCE DIAGRAM FOR ARTICLES MANAGEMENT 59

FIGURE (4.21) CLASS DIAGRAM 60

FIGURE (4.22) COMPONENT DIAGRAM 61

FIGURE (4.23) ERD 62
FIGURE (4.24) SITEMAP 64

List Of Tables

TABLE (2.1) COMPARATIVE ANALYSIS OF PREVIOUS WORKS AND PROPOSED SYSTEM 12
TABLE (3.1) NON-FUNCTIONAL REQUIREMENTS SPECIFICATION 18
TABLE (3.2) FUNCTIONAL REQUIREMENTS OF THE CURRENT PROJECT 19
TABLE (3.3) RISKS AND MITIGATION PLANS 23
TABLE (4.1) ARCHITECTURE DIAGRAM KEY / LEGEND 29
TABLE (4.2) USE CASE DIAGRAM KEY / LEGEND 31
TABLE (4.3) VIEW TEST REQUESTS USE CASE 31
TABLE (4.4) CHANGE TEST REQUEST STATUS USE CASE 32
TABLE (4.5) ADD A TEST REQUEST USE CASE 33
TABLE (4.6) EDIT A TEST REQUEST USE CASE 34
TABLE (4.7) ENTER TEST RESULT USE CASE 35
TABLE (4.8) ADD A MEDICAL RECORD USE CASE 36
TABLE (4.9) UPDATE MEDICAL RECORD DATA USE CASE 37
TABLE (4.10) ADD A DIAGNOSIS USE CASE 38
TABLE (4.11) UPDATE A DIAGNOSIS USE CASE 39
TABLE (4.12) ADD AN ARTICLE USE CASE 40
TABLE (4.13) EDIT AN ARTICLE USE CASE 41
TABLE (4.14) DELETE AN ARTICLE USE CASE 42
TABLE (4.15) CHANGE ARTICLE STATUS USE CASE 43
TABLE (4.16) ACTIVITY DIAGRAM KEY / LEGEND 51
TABLE (4.17) SEQUENCE DIAGRAM KEY / LEGEND 60
TABLE (4.20) CLASS DIAGRAM KEY / LEGEND 61
TABLE (4.21) COMPONENT DIAGRAM KEY / LEGEND 62
TABLE (4.22) ERD KEY / LEGEND 63
TABLE (4.23) SITEMAP KEY / LEGEND 65
TABLE (4.24) INITIAL UNIT TEST LIST 65
TABLE (4.25) REQUIREMENTS TRACEABILITY MATRIX 67
TABLE (5.1) IMPLEMENTATION ENVIRONMENT AND TOOLCHAIN 69
TABLE (5.2) REPOSITORY DIRECTORY STRUCTURE AND RESPONSIBILITIES 71

Chapter 1

introduction

1.1 Project Background:

The ongoing digital transformation across global service-oriented sectors has fundamentally reshaped how organizations manage operations, interact with users, and ensure service quality. In the healthcare domain, the paradigm shift from paper-based administrative and clinical workflows to integrated digital systems has become a strategic imperative. Primary health centers, serving as the foundational nodes of community healthcare delivery, handle a high volume of patient interactions, medical documentation, scheduling, and cross-departmental coordination.

Traditionally, many health centers, particularly in developing or resource-constrained environments, rely on fragmented, manual, or loosely integrated digital tools. These legacy processes frequently lead to data fragmentation, scheduling inefficiencies, delayed information retrieval, and elevated risks of administrative or clinical errors. To address these challenges, this graduation project introduces the **Health Center Management System (HCMS)**, a comprehensive web-based application designed to centralize and streamline operational, administrative, and clinical workflows. Developed under the Software Engineering and Information Systems curriculum, the system integrates core departments—including reception, medical records, laboratory, and accounting—into a unified digital environment, leveraging modern software engineering practices to enhance data accuracy and support evidence-based decision-making.

1.2 Problem Statement:

Despite the critical role of primary healthcare facilities, many health centers continue to suffer from severe operational inefficiencies due to inadequate digital infrastructure. The core problems addressed by this project include:

- **Data Fragmentation and Redundancy:** Patient records, appointment schedules, and financial invoices are often managed in isolated silos or paper formats, leading to redundant data entry and a high probability of human error.
- **Inefficient Clinical Workflows:** The lack of a centralized system for managing medical records, diagnostic test requests, and treatment histories delays patient care and disrupts communication between doctors and specialists.
- **Poor Administrative Oversight:** Management lacks real-time visibility into center operations, resulting in delayed statistical reporting, inefficient resource allocation, and unmonitored staff performance.
- **Inadequate Patient Engagement:** Existing systems often neglect integrated mechanisms for patient feedback, health education dissemination (e.g., medical articles), and satisfaction tracking.
- **Security and Privacy Vulnerabilities:** Manual or poorly integrated systems lack robust Role-Based Access Control (RBAC) and secure data storage, exposing sensitive medical information to unauthorized access.

These challenges directly affect all stakeholders, including patients (experiencing delayed care), medical staff (facing administrative burdens), and administrators (lacking actionable data).

1.3 Project Significance:

The proposed HCMS delivers substantial value on both practical and academic levels:

- **Practical Significance:** The system provides a centralized, web-accessible platform that unifies clinical and administrative functions. Practically, it reduces manual workload, minimizes operational

bottlenecks, and accelerates information retrieval. By offering native Arabic language support and an intuitive interface, it ensures high user acceptance among local healthcare providers, directly supporting the center's operational resilience and quality of care.

- **Academic Significance:** From an academic perspective, this project serves as a structured, end-to-end application of software engineering methodologies to a real-world health informatics problem. It demonstrates the practical implementation of requirements elicitation, iterative development cycles, Unified Modeling Language (UML) modeling, structured database normalization, and systematic testing, bridging the gap between theoretical computer science concepts and production-ready software.

1.4 Project Objectives:

Primary Objective: To design, develop, and validate a secure, scalable, and user-friendly web-based Health Center Management System that centralizes administrative and clinical operations, thereby improving workflow efficiency, data integrity, and service quality.

Secondary Objectives:

1. **Secure Access Control:** Implement a robust authentication mechanism and Role-Based Access Control (RBAC) tailored for five distinct user roles (Patient, Doctor, Specialist, Staff, Administrator).
2. **Clinical Workflow Optimization:** Streamline patient care by enabling efficient management of appointments, medical records, diagnostic test requests, and treatment histories (covering requirements REQ-042 to REQ-050).

3. **Administrative and Financial Oversight:** Equip staff and administrators with tools for department lifecycle management, invoice generation, real-time statistical dashboards, and structured report exportation.
4. **Patient Engagement and Education:** Support the publication and management of health articles, patient feedback surveys, and service ratings to monitor and enhance patient satisfaction (covering requirements REQ-051 to REQ-054).

1.5 Project Scope:

Inclusions: The project encompasses the complete Software Development Life Cycle (SDLC) for a web-based HCMS. It covers the analysis, design, implementation, and testing of **54 functional requirements (REQ-001 to REQ-054)**. The system supports five primary actor roles and integrates modules for user management, clinical records, laboratory test requests, appointment scheduling, content publishing, feedback collection, and financial invoicing. The technical scope includes a Client-Server architecture utilizing PHP for backend logic, MySQL for relational database management, and HTML/CSS/JavaScript for the frontend, deployed and tested within a local XAMPP environment.

Exclusions (Constraints):

- The system is developed as an academic prototype; therefore, enterprise-grade load balancing, cloud-native auto-scaling, and formal regulatory compliance (e.g., HIPAA/GDPR) are out of scope for this phase.
- Advanced features such as telemedicine integration, AI-driven predictive analytics, and native mobile applications are architecturally

accommodated for future work but are excluded from the current implementation scope.

1.6 Project Contributions:

This project delivers distinct contributions to the field of software engineering and health informatics:

- **Practical Contribution:** It provides a cost-effective, modular, and easily maintainable digital solution tailored for mid-sized health centers. By unifying disparate workflows into a single platform with native Arabic localization, it significantly reduces administrative overhead and enhances patient care continuity.
- **Technical and Academic Contribution:** The project demonstrates a rigorous application of iterative software development, requirements traceability (via a Requirements Traceability Matrix), and model-driven design. It establishes a reproducible academic reference and a foundational architecture that can be extended to incorporate machine learning, IoT-enabled medical devices, or cloud-based microservices in subsequent research iterations.

1.7 Report Organization:

This report is systematically structured to guide the reader through the complete software engineering lifecycle of the Health Center Management System. The document is organized into seven sequential chapters, followed by references and appendices, as outlined below:

- **Chapter 1: Introduction** – Presents the project background, problem statement, significance, objectives, scope, and contributions.

- **Chapter 2: Fundamental Concepts and Reference Studies** – Explains core theoretical concepts, reviews recent reference studies and previous works, compares existing systems, and identifies the research gap.
- **Chapter 3: Project Methodology and Requirements Analysis** – Details the adopted development methodology, outlines functional and non-functional requirements, assesses feasibility and risks, and presents the project schedule.
- **Chapter 4: Analytical and Design Study** – Presents the high-level architecture, UML diagrams (Use Case, Activity, Sequence, Class, Component), Entity-Relationship Diagrams (ERD), Sitemap, and the Requirements Traceability Matrix.
- **Chapter 5: Implementation** – Describes the execution environment, software project structure, module implementation, API integration, and deployment protocols.
- **Chapter 6: Testing and Evaluation** – Documents the testing strategy, test cases and results, performance evaluation, and the final update of the requirements traceability.
- **Chapter 7: Conclusion and Future Work** – Summarizes the project achievements, discusses encountered constraints, and proposes actionable pathways for future system enhancements.

Chapter 2
Fundamental Concepts and Reference Studies

2.1 Fundamental Concepts:

This section outlines the core theoretical concepts, terminologies, and architectural paradigms that form the foundation of the proposed Health Center Management System (HCMS).

- **Health Center Management System (HCMS):** A centralized, web-based software platform designed to automate, integrate, and streamline clinical, administrative, and financial workflows within a primary healthcare facility. It serves as a single source of truth for patient data, scheduling, and resource allocation.
- **Role-Based Access Control (RBAC):** A security paradigm that restricts system access based on predefined organizational roles. In the context of this project, RBAC ensures that the 54 functional requirements (REQ-001 to REQ-054) are executed only by authorized actors (Patient, Doctor, Specialist, Staff, Administrator), thereby safeguarding sensitive medical data.
- **Client-Server Architecture & MVC Pattern:** The system utilizes a distributed computing model where client devices (web browsers) request services from a centralized server (Apache/PHP). Conceptually, it adheres to the Model-View-Controller (MVC) pattern to separate data logic (Model), user interface (View), and request processing (Controller), ensuring modularity and maintainability.
- **Requirements Traceability Matrix (RTM):** A structured documentation tool that maps each functional and non-functional requirement to its corresponding design artifacts, source code modules, and test cases. The RTM guarantees that all 54 requirements are fully covered and validated throughout the Software Development Life Cycle (SDLC).

- **Entity-Relationship Modeling (ERM) & Normalization:** A conceptual data modeling approach used to design the relational database (MySQL). The system applies normalization principles (up to Third Normal Form - 3NF) to eliminate data redundancy, enforce referential integrity, and optimize query performance across interconnected entities (e.g., Patients, Appointments, Medical Records, Invoices).

2.2 Reference Studies and Previous Works:

The digitalization of primary healthcare has been the subject of extensive recent research and industrial development. To establish a baseline for the proposed architecture, this study reviews prominent contemporary health information systems and recent academic literature in the field of health informatics.

1. **OpenMRS [1]:** OpenMRS is a globally recognized, open-source electronic medical record (EMR) platform designed primarily for resource-constrained environments. Recent evaluations highlight its robust patient information management and modular architecture, which supports integration via RESTful APIs. However, its enterprise-grade complexity and reliance on specialized Java-based middleware necessitate significant technical expertise for deployment, making it less viable for small-to-mid-sized local clinics.
2. **GNU Health [2]:** GNU Health is a comprehensive, free health and hospital information system built on a Python/PostgreSQL stack. Recent studies emphasize its extensive capabilities in chronic disease tracking, immunization registries, and integrated financial modules. Despite its scalability for large governmental institutions, its heavy dependency on Linux-based server environments and complex resource management present a steep learning curve and high infrastructure costs for mid-sized health centers.

3. **ClinicMaster [3]:** ClinicMaster represents the commercial Software-as-a-Service (SaaS) approach to clinic management. It streamlines patient scheduling, invoicing, and statistical reporting through a highly intuitive, user-centric interface. While it excels in ease of use, its proprietary nature, vendor lock-in, and lack of native Arabic localization restrict its accessibility and adaptability for academic or non-profit healthcare deployments in the region.
4. **Recent Web-Based Health Informatics Trends [4]:** Contemporary academic research (2021-2024) emphasizes a paradigm shift toward lightweight, web-based Client-Server architectures utilizing PHP/MySQL or similar accessible stacks. Studies indicate that mid-sized health centers prioritize systems that offer rapid deployment, low maintenance costs, and integrated patient engagement tools (e.g., feedback loops, health education portals) over heavy, monolithic enterprise systems.

2.3 Comparison of Previous Works:

To critically evaluate the existing landscape, a structured comparative analysis was conducted. Table 2.1 highlights the key differentiators in terms of technologies, features, and strengths among the reference systems and the proposed HCMS.

Table (2.1) Comparative Analysis of Previous Works and Proposed System

Study / System	Technologies Used	Feature 1	Feature 2	Strengths
OpenMRS [1]	Java, MySQL, REST APIs	Modular EHR & Clinical Notes	Community-driven API ecosystem	Highly adaptable, open-source, globally validated.
GNU Health [2]	Python, PostgreSQL, Tryton	Full Hospital Info System	Integrated Financial & Inventory	Comprehensive, highly scalable for large government networks.
ClinicMaster [3]	Proprietary SaaS, Cloud	Intuitive UI & Scheduling	Automated Billing & Reporting	User-friendly, zero server maintenance, rapid deployment.
Proposed HCMS	PHP, MySQL, Apache (XAMPP)	Native Arabic RTL Support	Integrated Patient Feedback Loop	Cost-effective, localized, lightweight, and academically traceable (REQ-001 to REQ-054).

2.4 Research Gap:

Despite the proliferation of health management platforms, a critical analysis of the reference studies and current market offerings reveals distinct gaps that this project aims to bridge:

1. **Deployment & Cost Barriers for Mid-Sized Centers:**
    - The Gap: Enterprise-grade systems like OpenMRS and GNU Health require substantial IT infrastructure, specialized administrative overhead, and complex dependency management, rendering them inaccessible for small-to-mid-sized health centers

in developing regions. Conversely, commercial SaaS products incur recurring licensing fees.

- The Solution: The proposed HCMS utilizes a lightweight, open-source stack (PHP/MySQL/Apache) that can be deployed on minimal hardware (e.g., local XAMPP environments or basic shared hosting), ensuring zero licensing costs and minimal technical barriers to entry.

2. **Fragmented Patient Engagement & Education:**
    - The Gap: Most reference systems focus heavily on internal clinical and administrative workflows (EHR, billing) while neglecting integrated mechanisms for patient engagement, such as health education dissemination, satisfaction tracking, and feedback loops.
    - The Solution: This project explicitly addresses this gap by integrating patient-centric modules (REQ-008 to REQ-010, REQ-026 to REQ-028, REQ-051 to REQ-054). It includes functionalities for publishing medical articles, managing interactive surveys, and collecting service ratings, thereby fostering a transparent communication channel between the center and the community.
3. **Localization & Regional UX Limitations:**
    - The Gap: Globally distributed HCMS platforms often lack native Arabic language support, resulting in poor Right-to-Left (RTL) interface optimization, which severely hinders adoption among local administrative staff and patients.
    - The Solution: The proposed system is engineered with native Arabic localization from the ground up. The frontend architecture

enforces RTL layout consistency, localized form validation, and region-specific administrative reporting, ensuring high user acceptance and operational efficiency.

4. **Academic-Industry Translation & Traceability:**
    - The Gap: Many academic graduation projects fail to demonstrate end-to-end requirement traceability, often resulting in "black-box" implementations where the mapping between initial stakeholder needs and final code is lost.
    - The Solution: This project enforces strict software engineering discipline by mapping all 54 functional requirements (REQ-001 to REQ-054) through a comprehensive Requirements Traceability Matrix (RTM), linking every feature to specific UML diagrams, database entities, and test cases (TC-01 to TC-55), serving as a reproducible academic reference.

Chapter 3

Project Methodology and Requirements Analysis

3.1 Adopted Development Methodology:

The project adopts the Iterative and Incremental Development Methodology. This approach was selected because it structures the software lifecycle into repeated cycles (iterations), where each cycle delivers a functional subset of the system's requirements. This methodology is highly suitable for this project for the following reasons:

1. **Progressive Validation:** It enables continuous verification of implemented requirements against stakeholder expectations, reducing the risk of late-stage functional mismatches.
2. **Risk Mitigation:** Early deployment of core modules (e.g., authentication, patient profiles) allows for rapid identification and resolution of technical bottlenecks before scaling to complex clinical modules.
3. **Adaptive Integration:** It aligns seamlessly with the structured delivery of the 54 functional requirements, which were grouped into development sprints based on dependency mapping.
4. **Academic Compliance:** It matches the pedagogical objectives of the Software Engineering curriculum by demonstrating model-driven design, requirements traceability, and empirical testing within a constrained academic timeframe.

3.2 General Functional Requirements of the System:

The general functional requirements describe the core capabilities the system must provide to automate and integrate the health center's operations, without delving into the granular technical implementation of each module. The system must support:

- **User Authentication & Role-Based Access Control (RBAC):** Secure login, session management, and dynamic routing tailored to five distinct roles (Patient, Doctor, Specialist, Staff, Administrator).
- **Clinical & Patient Data Management:** Centralized storage and retrieval of patient profiles, medical histories, diagnostic records, and laboratory test requests.
- **Appointment Scheduling & Tracking:** End-to-end management of patient appointments, including booking, modification, cancellation, and real-time status tracking.
- **Administrative & Financial Oversight:** Tools for managing organizational departments, generating and exporting financial invoices, and viewing real-time statistical dashboards.
- **Patient Engagement & Content Management:** Mechanisms for publishing health articles, managing interactive patient surveys, and collecting service ratings to foster a transparent feedback loop.

3.3 Non-Functional Requirements of the System:

Non-functional requirements define the quality attributes, security constraints, and performance standards of the system.

Table (3.1) Non-Functional Requirements Specification

Code	Description	Non-functional Requirement
NFR-01	Usability	The system interface must be simple, intuitive, and consistent across all modules to minimize user error.
NFR-02	Localization	The system must fully support the Arabic language (RTL layout, Arabic fonts, and localized labels).
NFR-03	Accuracy	The system must perform calculations (e.g., invoices, statistics) with 100% accuracy and validate all input data formats.
NFR-04	Availability	The system must be accessible 24/7 via web browser without significant downtime during normal operations.
NFR-05	Performance	Page load times and database query responses must remain under 2 seconds for standard operations.
NFR-06	Security	User passwords must be encrypted (using secure hashing algorithms) before storage in the database.
NFR-07	Security	Access to all pages must be protected by session validation; unauthorized access attempts must be redirected to login.
NFR-08	Security	Role-Based Access Control (RBAC) must strictly enforce permissions; users can only access features relevant to their role.
NFR-09	Scalability	The database schema and code structure must allow for future module additions without major refactoring.
NFR-10	Integration	Data entered once (e.g., patient registration) must be automatically available across all related modules.

3.4 Functional Requirements of the Current Project

The functional requirements define the specific behaviors and functions the system must support. They are mapped to unique identifiers (REQ-001 to REQ-054) for traceability. Note: Requirements REQ-042 to REQ-054 represent the specialized clinical and content management modules developed in the advanced phases of this project.

Table (3.2) Functional Requirements of the Current Project

Priority	Code	Requirement	Description
1	REQ-001	Log in	Log in to each rank of users and direct each user to his own pages.
1	REQ-002	Edit Profile	The user has the ability to modify his personal data such as e-mail and password.
1	REQ-003	View profile	Viewing user's personal account information.
1	REQ-004	View doctors list	View the list of registered doctors at the center.
1	REQ-005	View specialists list	View the list of specialists registered at the center.
1	REQ-006	View staff list	View the list of center employees.
1	REQ-007	View departments list	View the list of departments within the center.
1	REQ-008	View articles	View articles listed by doctors or specialists.
1	REQ-009	View ratings	View reviews submitted by patients.
1	REQ-010	View surveys	View questionnaires added by doctors, specialists or employees.
1	REQ-011	Recover password	Password recovery for system users in case they forget it.
2	REQ-012	Manage system settings	Manage system settings such as dark mode or change billing settings.
2	REQ-013	View statistics	View comprehensive statistics on operations within the system.
2	REQ-014	Export reports	Export reports on the work of the system.
2	REQ-015	View patient list	View the list of patients registered in the system.

2	REQ-016	Manage doctors' accounts	Manage the accounts of doctors (approval, rejection, deactivation).
2	REQ-017	Manage specialists' accounts	Managing the accounts of specialists (approval, rejection, deactivation).
2	REQ-018	Manage staff accounts	Manage the accounts of employees (approval, rejection, deactivation).
2	REQ-019	View medical record	View patients' medical records.
2	REQ-020	Create user accounts	Create new accounts for users who want to use the system.
2	REQ-021	Book an appointment	Book a new appointment within the center.
2	REQ-022	Modify an appointment	Modification of a pre-booked appointment within the center.
2	REQ-023	Cancel an appointment	Cancel a pre-booked appointment.
2	REQ-024	Track appointment status	Track the status of an appointment to see its current state.
3	REQ-025	View appointment history	View the history of registered appointments.
3	REQ-026	Submit a survey	Providing answers to the questionnaires listed by employees.
3	REQ-027	Add a rating	Add an evaluation of a service within the center by the patient.
3	REQ-028	Update a rating	Update a previously added rating.
3	REQ-029	Add a department	Adding a new section to the center's sections.
3	REQ-030	Edit department data	Modify the data of a previously listed section within the center.
3	REQ-031	Delete a department	Delete a section from the pre-existing sections in the center.

3	REQ-032	Change department status	Changing the status of a department (active, discontinued).
3	REQ-033	Add an invoice	Adding a new invoice by the employee.
3	REQ-034	Edit an invoice	Modify a previously added invoice.
4	REQ-035	Export an invoice	Export the invoice to be printable.
4	REQ-036	View invoice list	View the list of invoices included in the system.
4	REQ-037	Add a survey	Adding a new questionnaire.
4	REQ-038	Edit a survey	Edit questionnaire questions.
4	REQ-039	Delete a survey	Delete a questionnaire.
4	REQ-040	Change survey status	Changing the status of a questionnaire (active, hidden).
4	REQ-041	View survey responses	View responses to a pre-listed questionnaire.
4	**REQ-042**	**View test requests**	**View analysis requests submitted by doctors.**
4	**REQ-043**	**Change test request status**	**Change the status of an analysis request by the specialist.**
4	**REQ-044**	**Add a test request**	**Add an analysis request for a patient by a doctor.**
4	**REQ-045**	**Edit a test request**	**Modification of a request for additive analysis of a patient.**
4	**REQ-046**	**Enter test result**	**Entering the result of an analysis by the responsible specialist.**
4	**REQ-047**	**Add a medical record**	**Adding a new patient medical record.**

4	**REQ-048**	**Update medical record data**	**Updating a pre-listed medical record.**
5	**REQ-049**	**Add a diagnosis**	**Add a new diagnosis to the patient's condition by the doctor.**
5	**REQ-050**	**Update a diagnosis**	**A pre-added diagnostic update by the doctor.**
5	**REQ-051**	**Add an article**	**Adding a new health article.**
5	**REQ-052**	**Edit an article**	**Edit a previously added article.**
5	**REQ-053**	**Delete an article**	**Delete a previously added article.**
5	**REQ-054**	**Change article status**	**Changing the status of an article (available, hidden).**

3.5 Feasibility and Risks

Feasibility Study

A comprehensive feasibility study was conducted to assess the viability of the project across four critical dimensions:

1. **Technical Feasibility:** The system is built upon mature, open-source technologies (PHP, MySQL, Apache) compatible with standard local environments (XAMPP). The Client-Server architecture ensures modularity and does not demand specialized hardware.
2. **Operational Feasibility:** The system directly addresses critical pain points in manual workflows (data redundancy, scheduling conflicts). The inclusion of native Arabic RTL support ensures high user acceptance among local healthcare providers.
3. **Economic Feasibility:** As an academic project utilizing open-source software (OSS), it incurs zero licensing costs. The primary investment is

human capital, making it a highly cost-effective digital transformation solution for mid-sized health centers.

4. **Academic Feasibility:** The project comprehensively covers multiple domains in Software Engineering and Information Systems, including: requirements engineering, system design (UML/ERD), relational database normalization, web development, systematic testing, and project management, fulfilling all academic graduation criteria.

Risks and Mitigation Plans

A proactive risk management framework was implemented to anticipate and mitigate potential challenges.

Table (3.3) Risks and Mitigation Plans

Risk	Impact	Probability	Mitigation Plan
Scope Creep & Requirement Drift	High	Medium	Strict adherence to the approved Requirements Traceability Matrix (RTM). New features are deferred to future work.
Integration Complexity & Data Redundancy	High	Medium	Early ERD modeling and relational schema normalization (3NF). Cross-module API calls validated via Sequence Diagrams.
Schedule Delays & Academic Deadlines	High	High	Time-buffer allocation within the Gantt chart, parallel task delegation, and biweekly supervisor review checkpoints.
Security & Privacy Vulnerabilities	Critical	Low	Implementation of password hashing, session timeout controls, RBAC on every route, and prepared SQL statements (PDO).
Performance Degradation Under Load	Medium	Low	Index optimization on frequently queried tables, pagination implementation, and local load testing prior to deployment.

3.6 Project Plan:

The project schedule was structured using a Gantt Chart-based timeline, dividing the development lifecycle into distinct phases with clearly defined milestones.

Milestones and Phases:

1. **Phase I: Reference Study & Requirements Elicitation** (Weeks 1-2): Literature review, stakeholder analysis, and formal documentation of REQ-001 to REQ-054.
2. **Phase II: Iteration 1 - Core Clinical Modules** (Weeks 3-4): Implementation of patient record viewing and initial clinical workflows.
3. **Phase III: Iteration 2 - Scheduling & Tracking** (Weeks 5-6): Appointment booking, modification, and real-time status tracking.
4. **Phase IV: Iteration 3 - Content & Feedback** (Weeks 7-8): Health articles, survey deployment, and service rating mechanisms.
5. **Phase V: Iteration 4 - Administrative & Financial** (Weeks 9-10): Department lifecycle management, invoice generation, and reporting.
6. **Phase VI: Iteration 5 - Advanced Clinical & Integration** (Weeks 11-12): Final integration of laboratory test requests (REQ-042 to REQ-046), advanced medical records, and cross-module synchronization.
7. **Phase VII: Documentation & Validation** (Weeks 13-16): Comprehensive technical writing, UML/ERD finalization, test case execution, and academic report compilation.

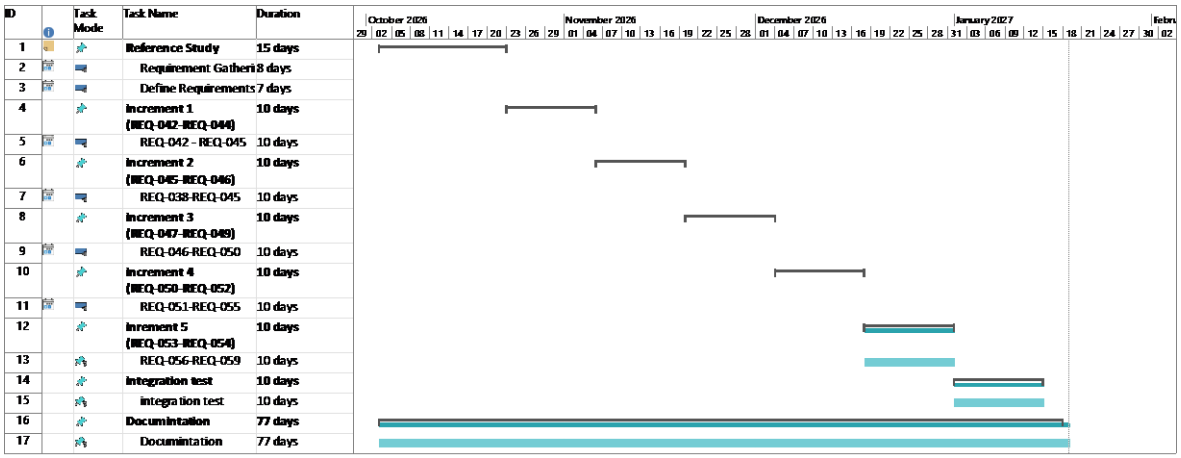


Figure (3.1) Gantt Chart

Task Distribution: The project was executed by a dedicated academic team under the supervision of Dr. Badr Ahmed:

- **Project Lead & System Architect:** Oversaw overall planning, iterative scheduling, UML modeling validation, and requirement traceability.
- **Database & Backend Engineer:** Responsible for ERD design, MySQL optimization, PHP backend logic, and secure API routing for all 54 requirements.
- **Frontend Developer & QA Specialist:** Designed responsive RTL user interfaces, implemented client-side validation, and executed systematic test case validation.

Chapter 4

Analytical and Design Study

4.1 System Architecture:

The HCMS is built upon a Three-Tier Client-Server Architecture integrated with the conceptual principles of the Model-View-Controller (MVC) pattern. This separation of concerns ensures modularity, security, and ease of maintenance.

1. **Presentation Tier (View):** HTML5/CSS3/JavaScript interfaces tailored for five distinct user roles, featuring native Arabic RTL support.
2. **Application Tier (Controller & Business Logic):** PHP-based server-side scripts handling session management, Role-Based Access Control (RBAC), and request processing.
3. **Data Tier (Model):** A centralized MySQL relational database ensuring data integrity, normalization (3NF), and secure storage of clinical and administrative records.

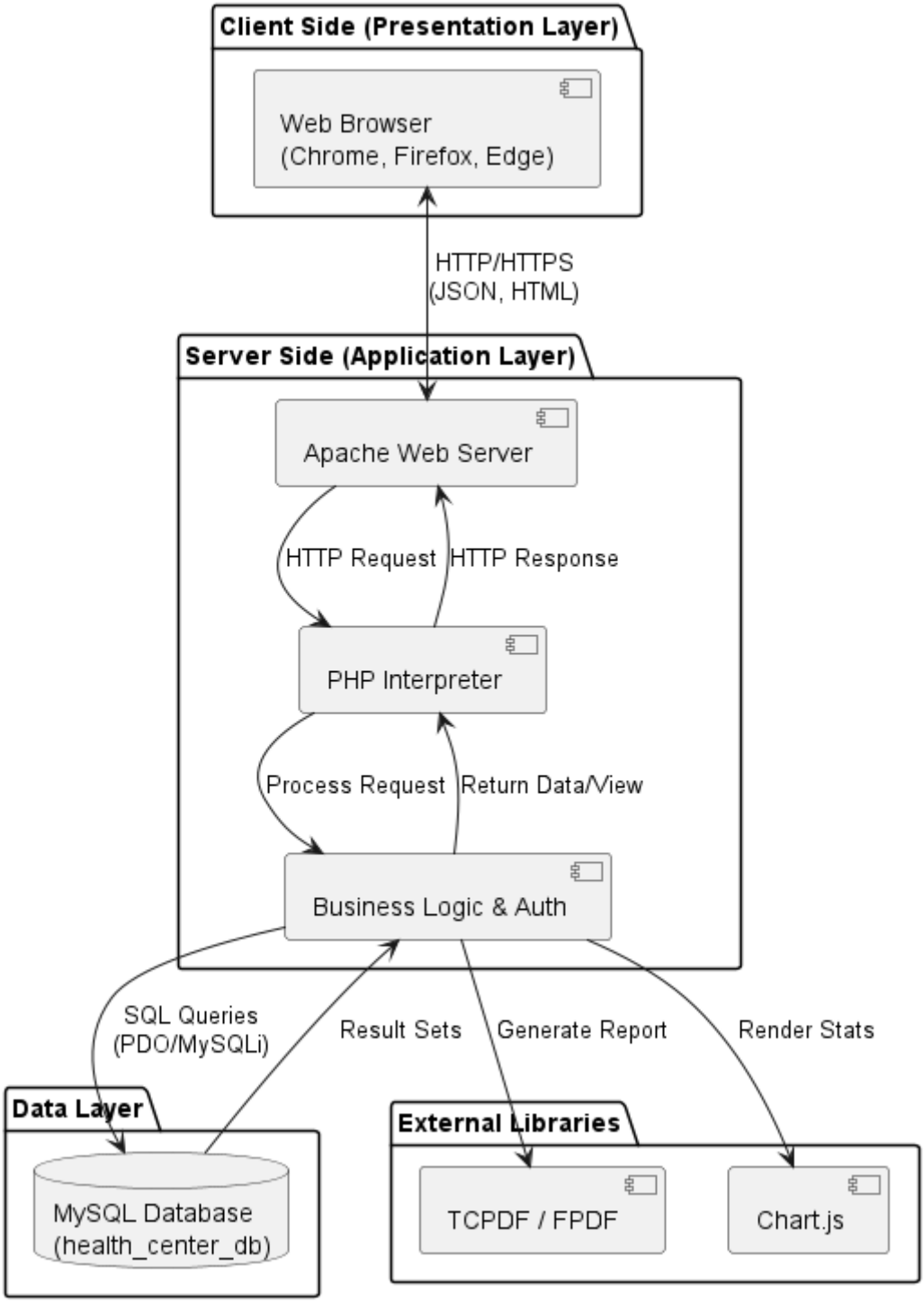


Figure (4.1) System Block Diagram

Table (4.1) Architecture Diagram Key / Legend

Symbol / Component	Description
Client Node	Represents the end-user's web browser (Chrome, Edge) rendering the UI.
HTTP/HTTPS	The communication protocol used for request-response cycles between client and server.
Apache Web Server	The server environment processing PHP scripts and serving static assets.
PHP Engine	Executes business logic, validates RBAC, and processes CRUD operations.
MySQL Database	The persistent storage layer managing relational data and enforcing constraints.

4.2 Use Case Diagram:

The Use Case Diagram illustrates the functional boundaries of the system and the interactions between the five primary actors (Patient, Doctor, Specialist, Staff, Administrator) and the system's core functionalities.



Figure (4.1) general use case diagram

Table (4.2) Use Case Diagram Key / Legend

Symbol	Description
Stick Figure	Represents an Actor (User or External System) interacting with the system.
Ellipse	Represents a Use Case (a specific system function or requirement).
Solid Line	Represents an association (communication) between an Actor and a Use Case.
System Boundary	The rectangular box defining the scope of the HCMS.
<<include>>	Indicates a mandatory sub-function (e.g., "Login" is included in all protected use cases).

4.2.1 Use Case Description:

Table (4.3) View test requests use case

ID	**REQ-042**
Use case	**View test requests**
Brief description	**Viewing analysis requests submitted by doctors for patients**
Actors	**Specialist, Doctor**
Previous conditions	**- User is logged in** **- Test requests exist in the system**
Subsequent terms	**- Successful completion of the operation**
The main flow	– **When the user selects the test requests option, the system displays the list of analysis requests submitted by doctors.** – **The system shows request details (patient name, test type, date, status)**
Alternative flow	**- When there are no test requests, a "No test requests" message appears.**

Table (4.4) Change test request status use case

ID	**REQ-043**
Use case	**Change test request status**
Brief description	**Changing the status of an analysis request by the specialist responsible**
Actors	**Specialist**
Previous conditions	**- Login** **- Test request exists**
Subsequent terms	**- Successful completion of the operation**
The main flow	**- When the specialist selects a test request, the system displays the request details.** **- The specialist changes the status (e.g., from Pending to In-Progress or Completed).** **- When the Save button is pressed, the system checks the data.** **- If valid, the system updates the request status in the database.**
Alternative flow	**- When there is an error, an error message is displayed asking the specialist to try again.**

Table (4.5) Add a test request use case

ID	**REQ-044**
Use case	**Add a test request**
Brief description	**Adding an analysis request for a patient by a doctor**
Actors	**Doctor**
Previous conditions	**- Login** **- Patient is selected**
Subsequent terms	**- Successful completion of the operation**
The main flow	**– When the doctor clicks on "Add Test Request", the system displays the request form.** **– The doctor fills in the required data (test type, clinical notes, urgency).** **– When the Save button is pressed, the system checks the entered data.** **– If the data is correct, the system saves the request and notifies the specialist.**
Alternative flow	**- When there is missing mandatory data, an error message is displayed asking the doctor to fill the indicated fields.**

Table (4.6) Edit a test request use case

ID	**REQ-045**
Use case	**Edit a test request**
Brief description	**Modification of an analysis request for a patient by a doctor**
Actors	**Doctor**
Previous conditions	**- Login** **- Test request exists and is not yet completed**
Subsequent terms	**- Successful completion of the operation**
The main flow	**– When the doctor selects a test request to edit, the system displays the request data.** **– The doctor modifies the required fields.** **– When the Save button is pressed, the system checks the data.** **– If correct, the system updates the request in the database.**
Alternative flow	**- If the request is already completed by the specialist, a message appears: "Cannot edit a completed request".**

Table (4.7) Enter test result use case

ID	**REQ-046**
Use case	**Enter test result**
Brief description	**Entering the result of an analysis by the responsible specialist**
Actors	**Specialist**
Previous conditions	**- Login** **- Test request is in "In-Progress" status**
Subsequent terms	**- Successful completion of the operation**
The main flow	**– When the specialist opens the test request, the system displays the result entry form.** **– The specialist enters the test results, values, and medical notes.** **– When the Save button is pressed, the system checks the data.** **– If correct, the system saves the result, updates the status to "Completed", and makes it visible to the doctor/patient.**
Alternative flow	**- When result data is missing, an error message is displayed asking to complete the required fields.**

Table (4.8) Add a medical record use case

ID	**REQ-047**
Use case	**Add a medical record**
Brief description	**Adding a new medical record for a patient**
Actors	**Doctor**
Previous conditions	**- Login** **- Patient profile is open**
Subsequent terms	**- Successful completion of the operation**
The main flow	**- When the doctor clicks on "Add Medical Record", the system displays the record form.** **- The doctor fills in the medical data (vitals, symptoms, general notes).** **- When the Save button is pressed, the system checks the data.** **- If correct, the system creates a new medical record linked to the patient's file.**
Alternative flow	**- When there is an error in the entered data, an error message is displayed asking the doctor to modify the fields.**

Table (4.9) Update medical record data use case

ID	**REQ-048**
Use case	**Update medical record data**
Brief description	**Updating a previously listed medical record**
Actors	**Doctor**
Previous conditions	**- Login** **- Medical record exists**
Subsequent terms	**- Successful completion of the operation**
The main flow	**− When the doctor selects a medical record, the system displays the record data.** **− The doctor modifies the required medical fields.** **− When the Save button is pressed, the system checks the data.** **− If correct, the system updates the medical record in the database.**
Alternative flow	**- If no changes were made, a message appears: "No changes to save".**

Table (4.10) Add a diagnosis use case

ID	**REQ-049**
Use case	**Add a diagnosis**
Brief description	**Adding a new diagnosis to the patient's condition by the doctor**
Actors	**Doctor**
Previous conditions	**- Login** **- Patient medical record is open**
Subsequent terms	**- Successful completion of the operation**
The main flow	**- When the doctor clicks on "Add Diagnosis", the system displays the diagnosis form.** **- The doctor enters the diagnosis details (name, ICD code, description, severity).** **- When the Save button is pressed, the system checks the data.** **- If correct, the system saves the diagnosis and links it to the patient's medical record.**
Alternative flow	**- When the diagnosis name is missing, an error message is displayed asking to fill the field.**

Table (4.11) Update a diagnosis use case

ID	**REQ-050**
Use case	**Update a diagnosis**
Brief description	**Updating a pre-added diagnosis by the doctor**
Actors	**Doctor**
Previous conditions	**- Login** **- Diagnosis exists in the patient record**
Subsequent terms	**- Successful completion of the operation**
The main flow	**- When the doctor selects a diagnosis, the system displays the diagnosis data.** **- The doctor modifies the diagnosis details or adds follow-up notes.** **- When the Save button is pressed, the system checks the data.** **- If correct, the system updates the diagnosis in the database.**
Alternative flow	**- When there is an error, an error message is displayed asking the doctor to correct the data.**

Table (4.12) Add an article use case

ID	**REQ-051**
Use case	**Add an article**
Brief description	**Adding a new health article by a doctor or specialist**
Actors	**Doctor, Specialist**
Previous conditions	**- Login**
Subsequent terms	**- Successful completion of the operation**
The main flow	**- When the user clicks on "Add Article", the system displays the article form.** **- The user fills in the article details (title, content, category, image).** **- When the Publish/Save button is pressed, the system checks the data.** **- If correct, the system saves the article and sets its initial status (Available/Hidden).**
Alternative flow	**- When the title or content is missing, an error message is displayed asking to fill the required fields.**

Table (4.13) Edit an article use case

ID	**REQ-052**
Use case	**Edit an article**
Brief description	**Editing a previously added article**
Actors	**Doctor, Specialist**
Previous conditions	**- Login** **- Article exists**
Subsequent terms	**- Successful completion of the operation**
The main flow	**- When the user selects an article to edit, the system displays the article data.** **- The user modifies the title, content, or image.** **- When the Save button is pressed, the system checks the data.** **- If correct, the system updates the article in the database.**
Alternative flow	**- When there is an error in the entered data, an error message is displayed asking to modify the fields.**

Table (4.14) Delete an article use case

ID	**REQ-053**
Use case	**Delete an article**
Brief description	**Deleting a previously added article**
Actors	**Doctor, Specialist**
Previous conditions	**- Login** **- Article exists**
Subsequent terms	**- Successful completion of the operation**
The main flow	**- When the user selects an article, the system displays the article options.** **- The user clicks the "Delete" button.** **- The system displays a confirmation message.** **- When the user confirms, the system deletes (or soft-deletes) the article from the database.**
Alternative flow	**- If the user cancels the deletion, the system returns to the articles list without making changes.**

Table (4.15) Change article status use case

ID	**REQ-054**
Use case	**Change article status**
Brief description	**Changing the status of an article (Available, Hidden)**
Actors	**Doctor, Specialist**
Previous conditions	**- Login** **- Article exists**
Subsequent terms	**- Successful completion of the operation**
The main flow	**– When the user views the articles list, the system displays the status toggle for each article.** **– The user changes the status (from Available to Hidden, or vice versa).** **– The system immediately updates the status in the database.** **– Hidden articles are no longer visible to patients.**
Alternative flow	**- None — status change is always permitted.**

4.3 Activity Diagram:

Activity Diagrams model the workflow and sequential logic of complex business processes.

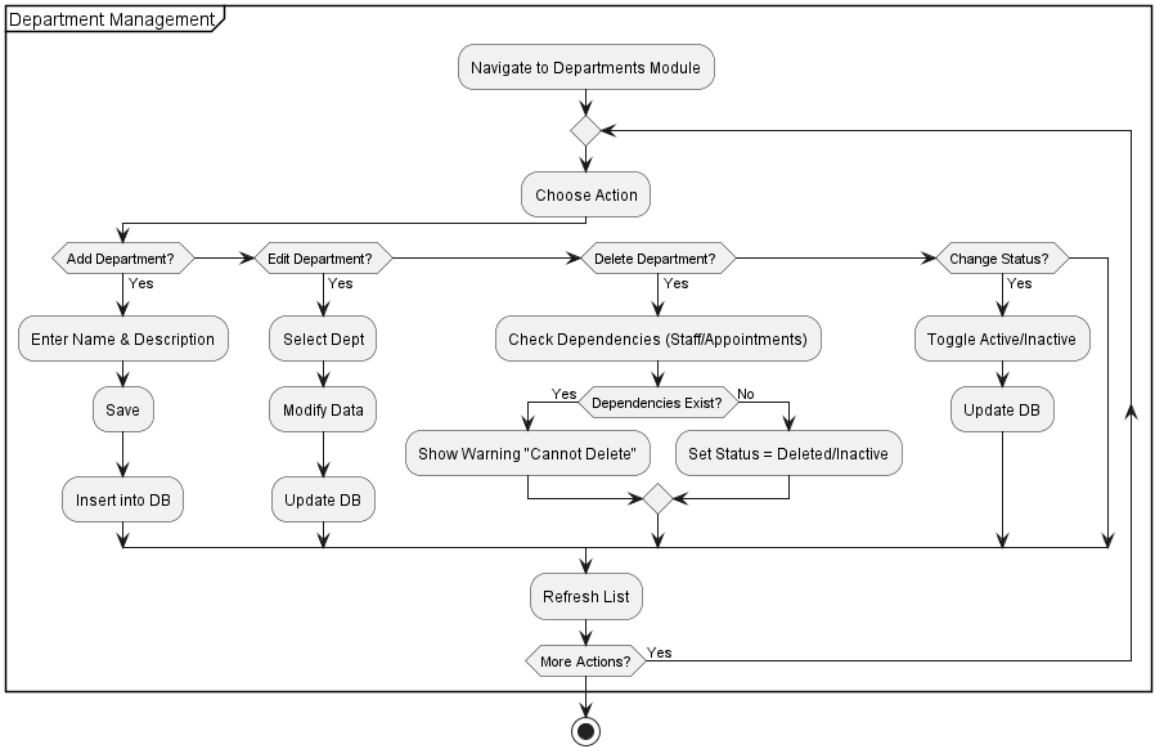


Figure (4.3) activity diagram for department management

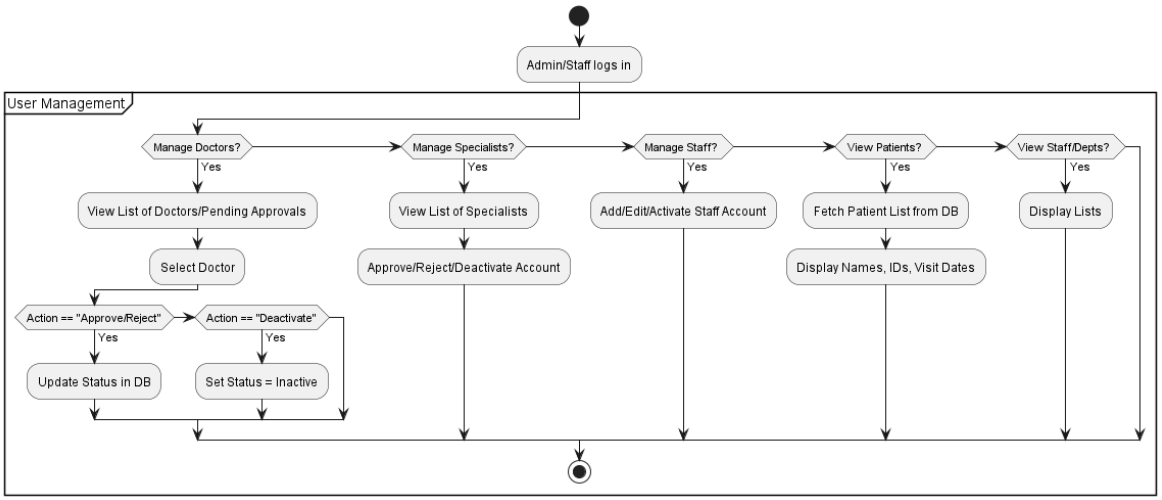


Figure (4.4) activity diagram for user management

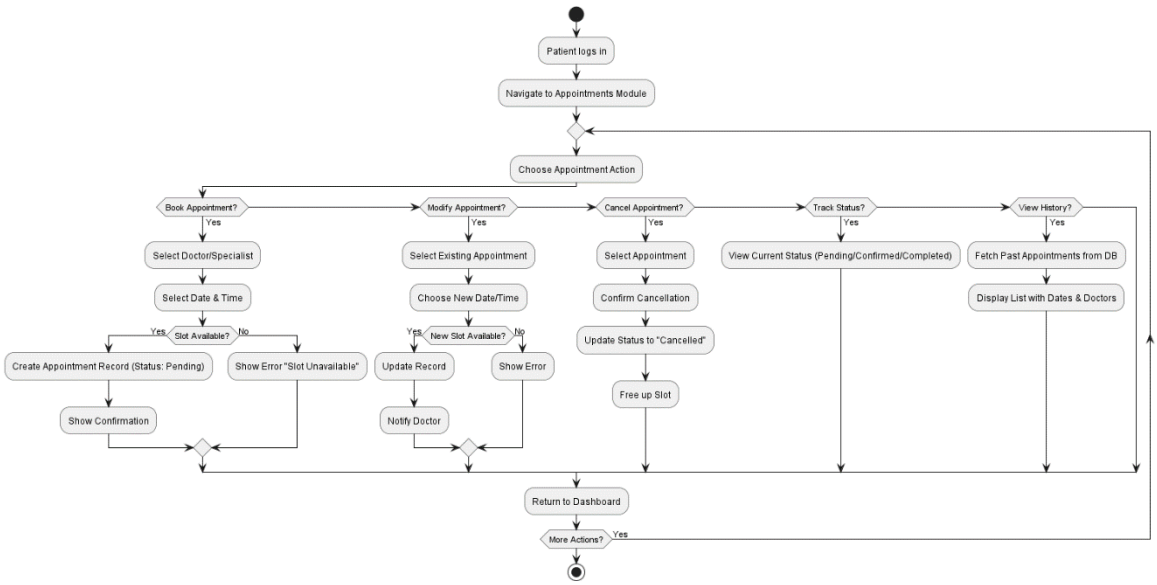


Figure (4.5) activity diagram for appointments management

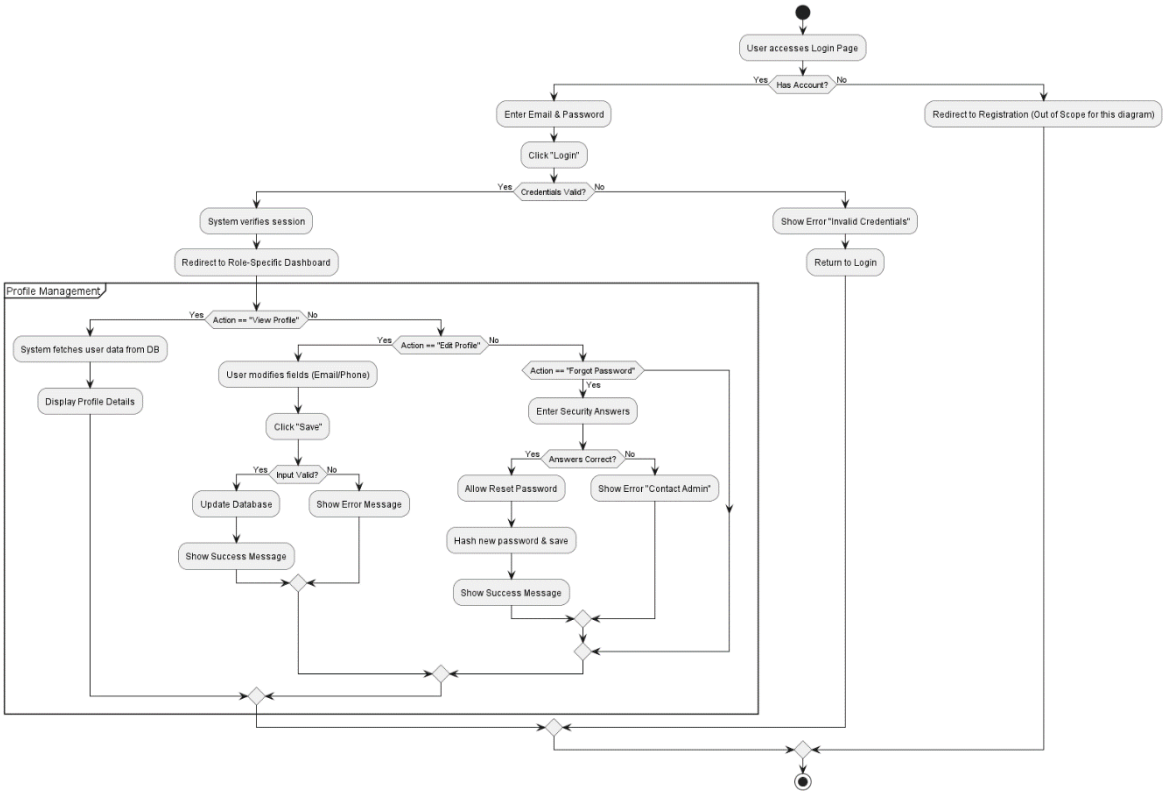


Figure (4.6) activity diagram for authentication and profile managements

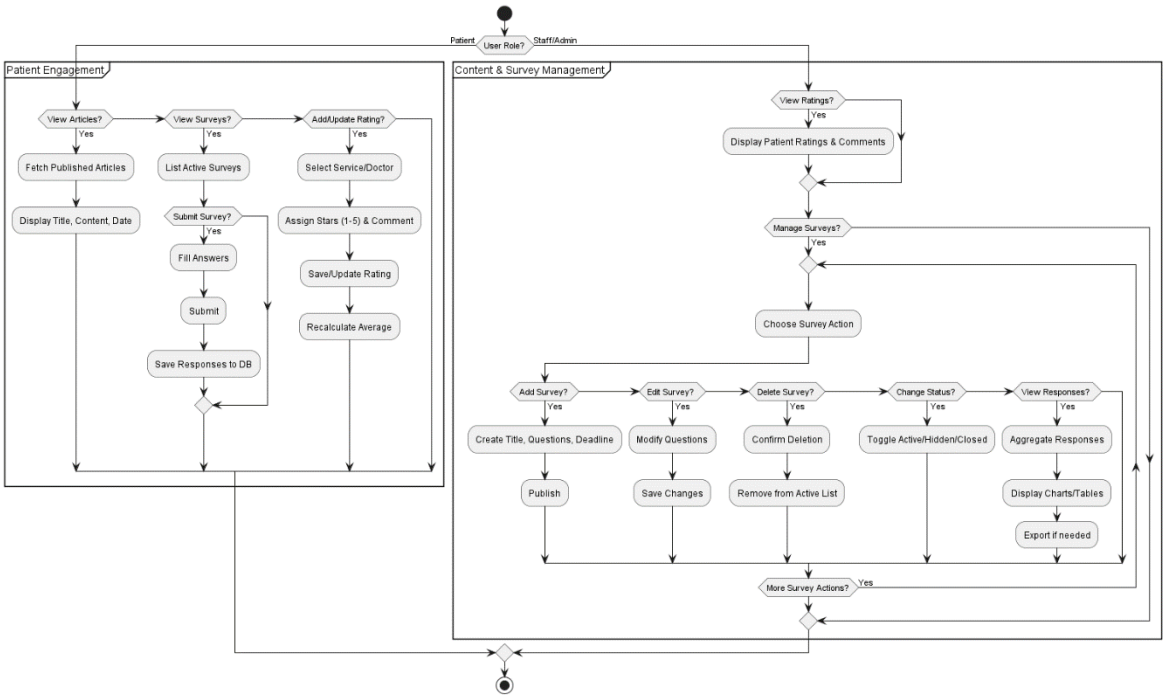


Figure (4.7) activity diagram for content and surveys and feedback

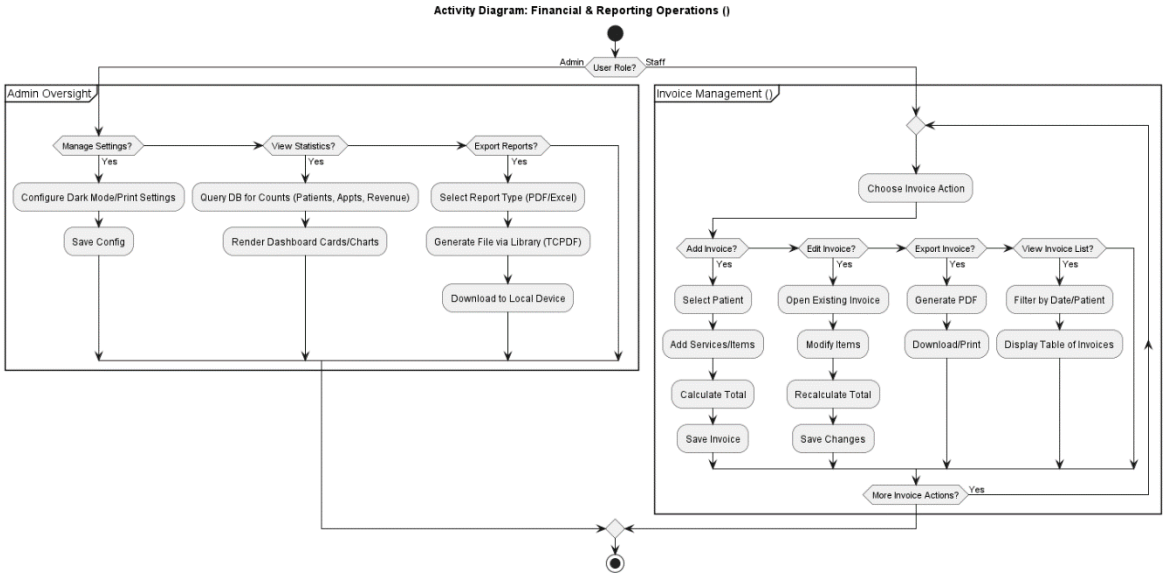


Figure (4.8) activity diagram for finical and reporting operation

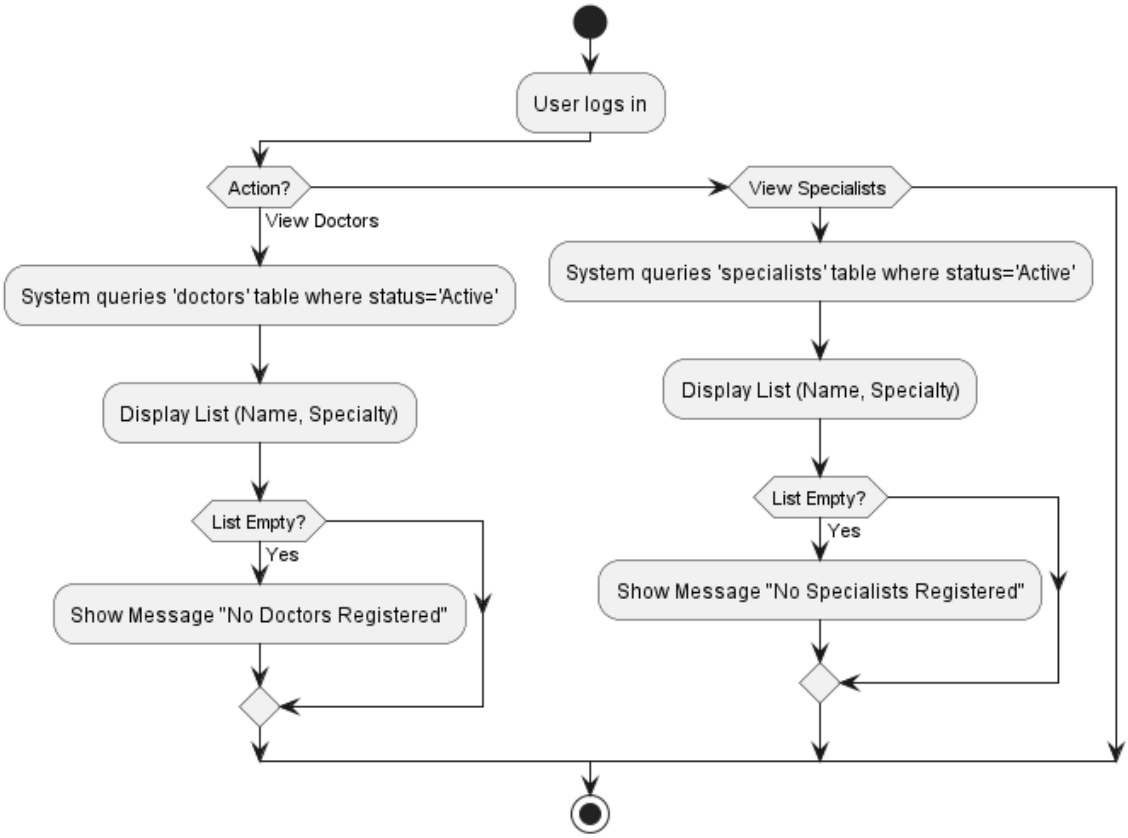


Figure (4.9) activity diagram for view doctors and specialists list

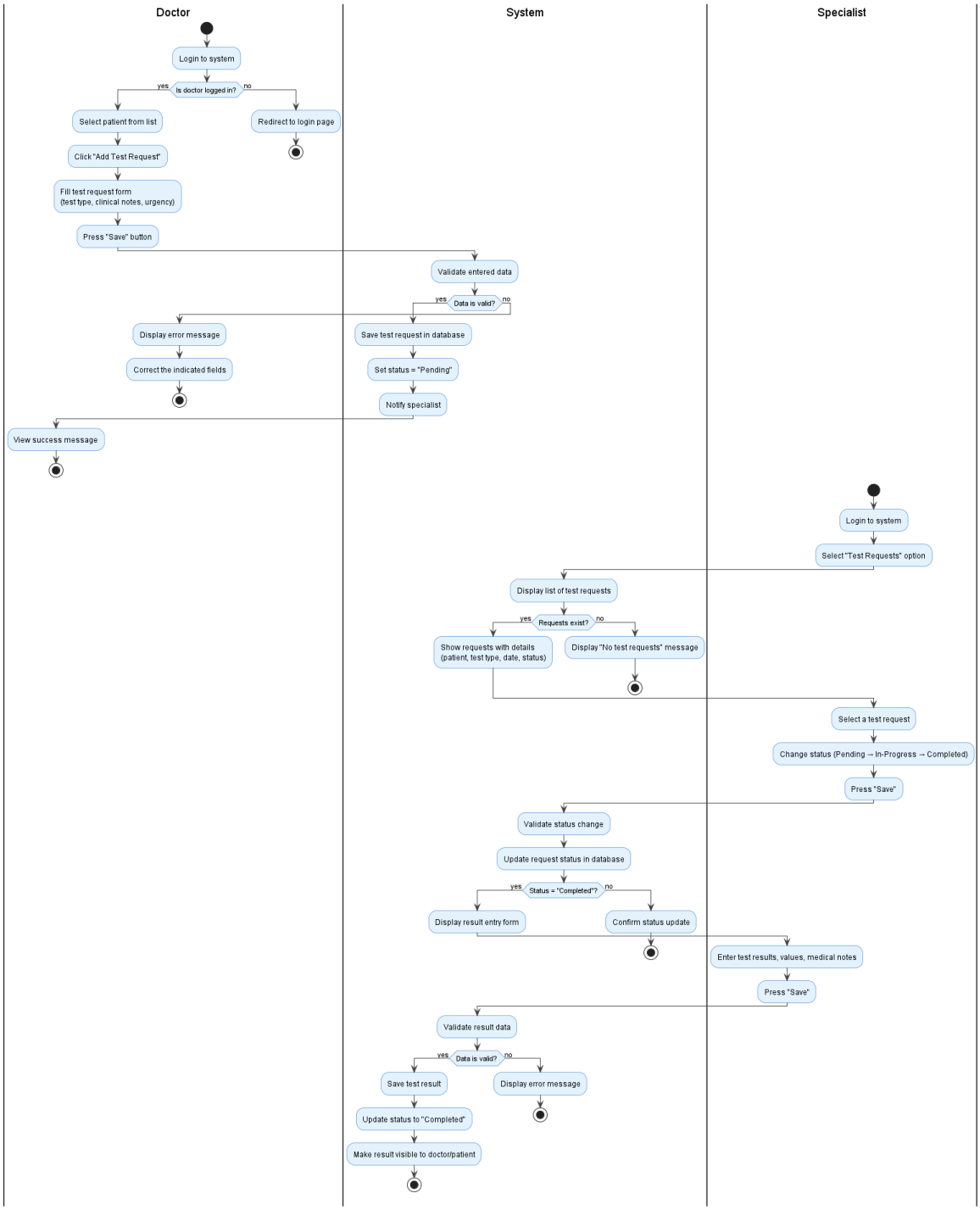


Figure (4.10) Activity Diagram for Test Requests Management

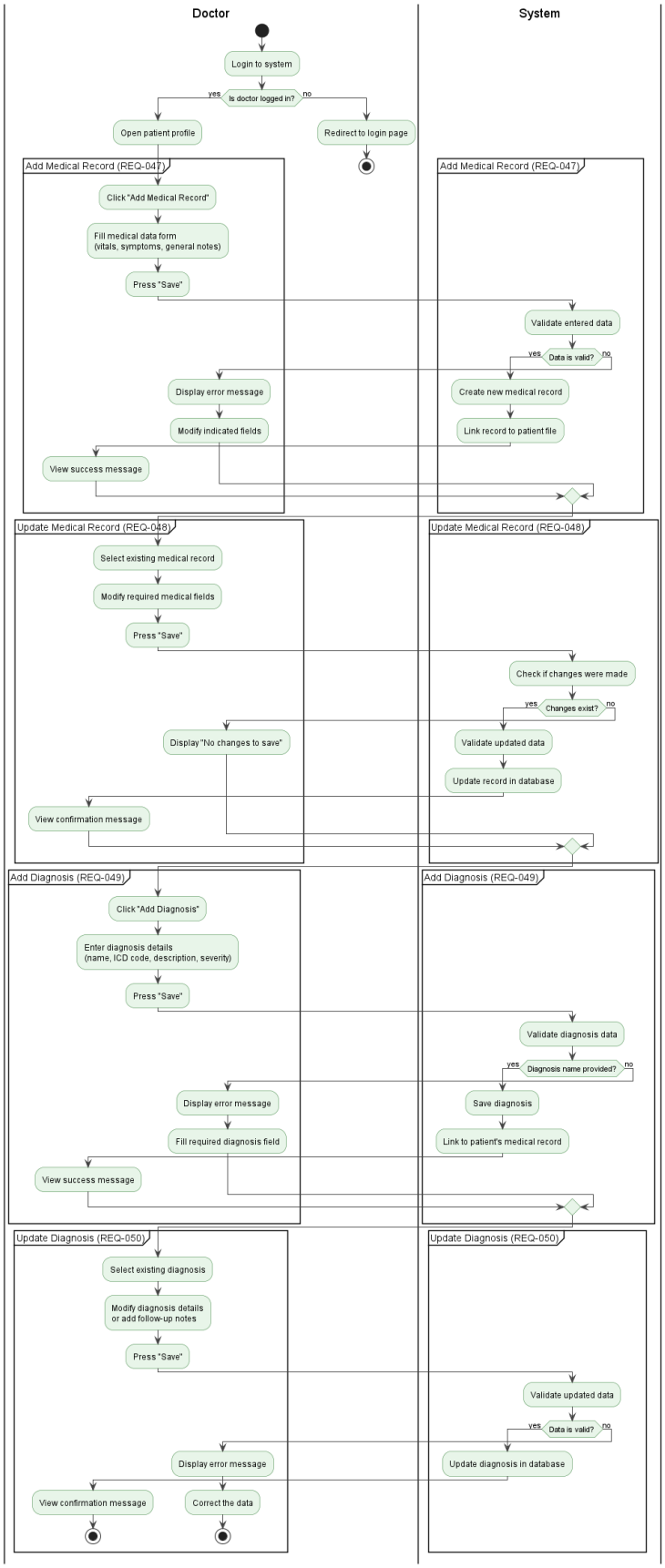


Figure (4.11) Activity Diagram for Medical Records & Diagnosis Management

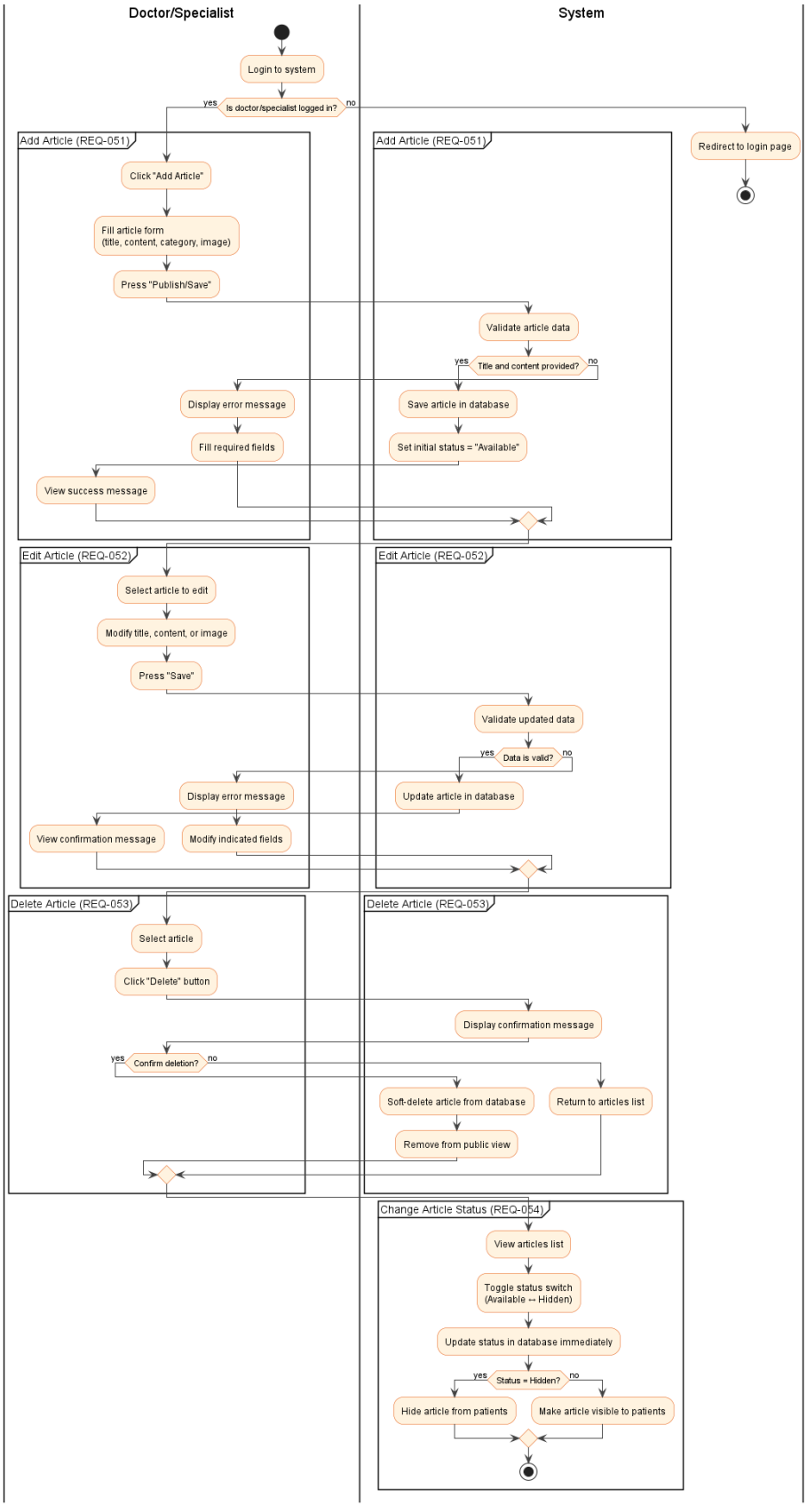


Figure (4.12) Activity Diagram for Articles Management

Table (4.16) Activity Diagram Key / Legend

Symbol	Description
Solid Circle	Initial Node (Start of the workflow).
Circle with Border	Final Node (End of the workflow).
Rounded Rectangle	Represents an Action or Activity performed by an actor or system.
Diamond	Decision Node (Conditional branching, e.g., "Is data valid?").
Vertical Bar (Thick)	Synchronization Bar (Fork/Join for concurrent activities).
Swimlane	Partitions the diagram to show which Actor/Department is responsible for each action.

4.4 Sequence Diagram:

Sequence Diagrams detail the object interactions arranged in time sequence. They depict the objects involved in the scenario and the sequence of messages exchanged.

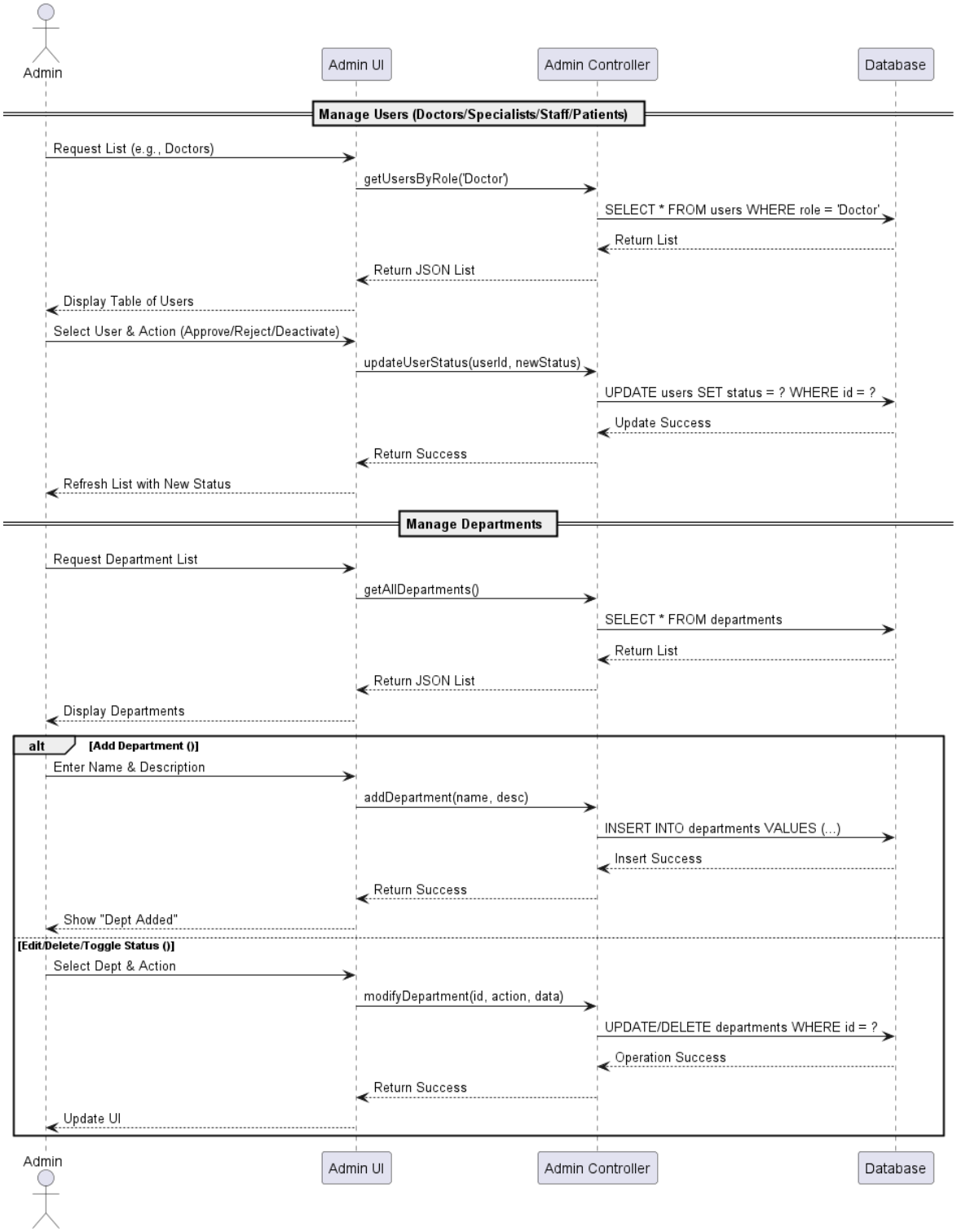


Figure (4.13) Sequence diagram for users and department managements

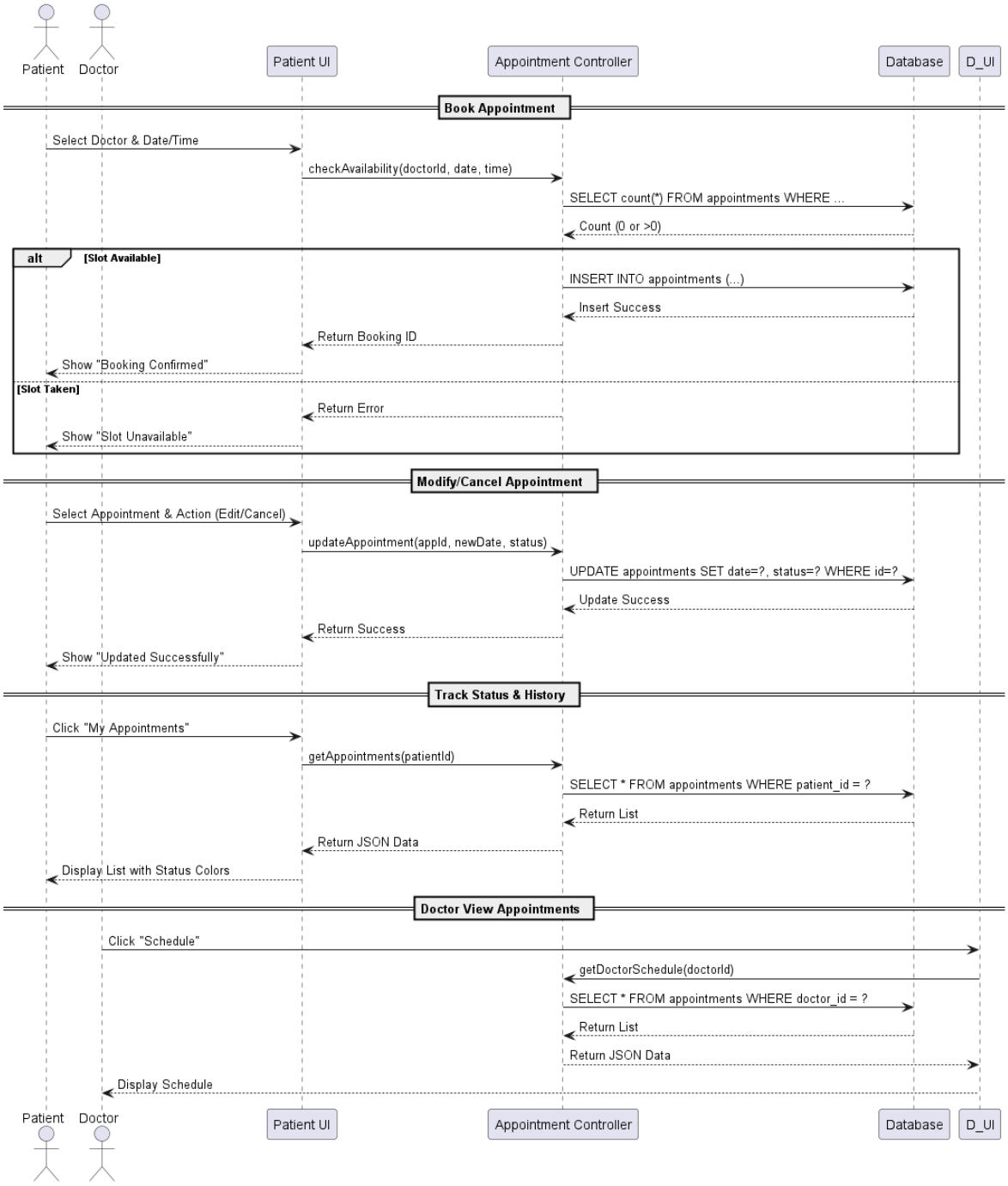


Figure (4.14) Sequence diagram for appointment managements

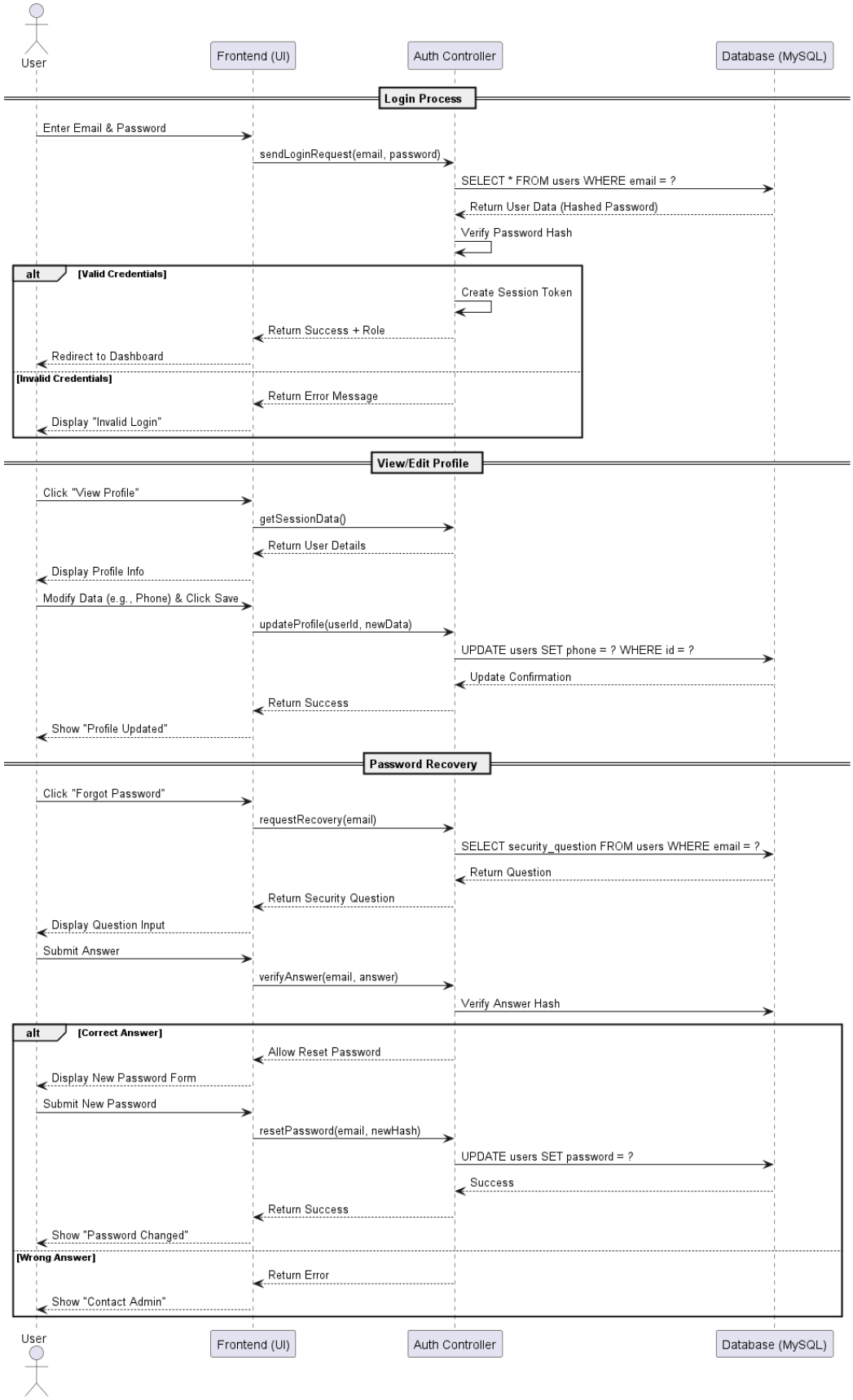


Figure (4.15) Sequence diagram for authentication and profile managements

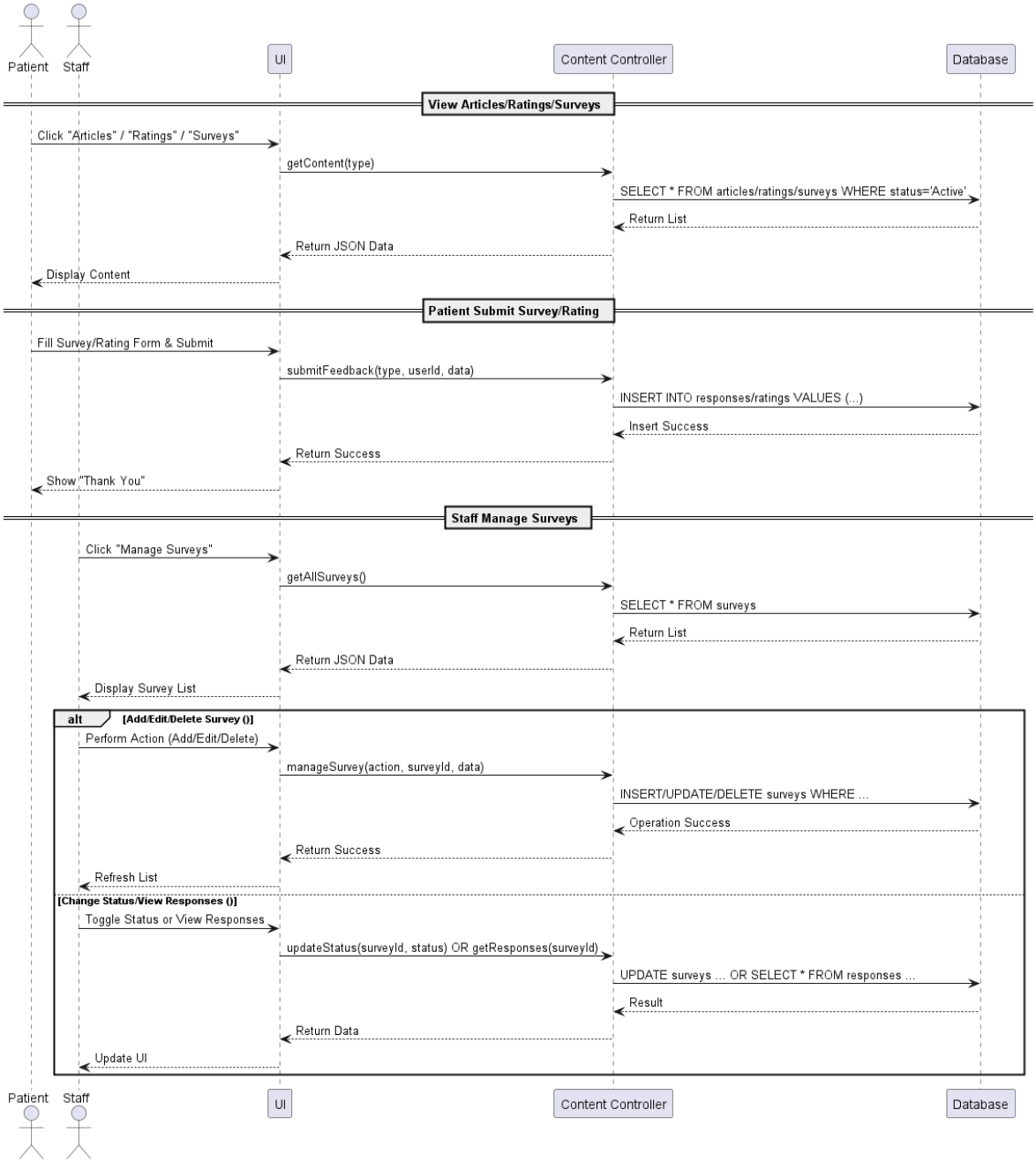


Figure (4.16) Sequence diagram for content and surveys and feedback

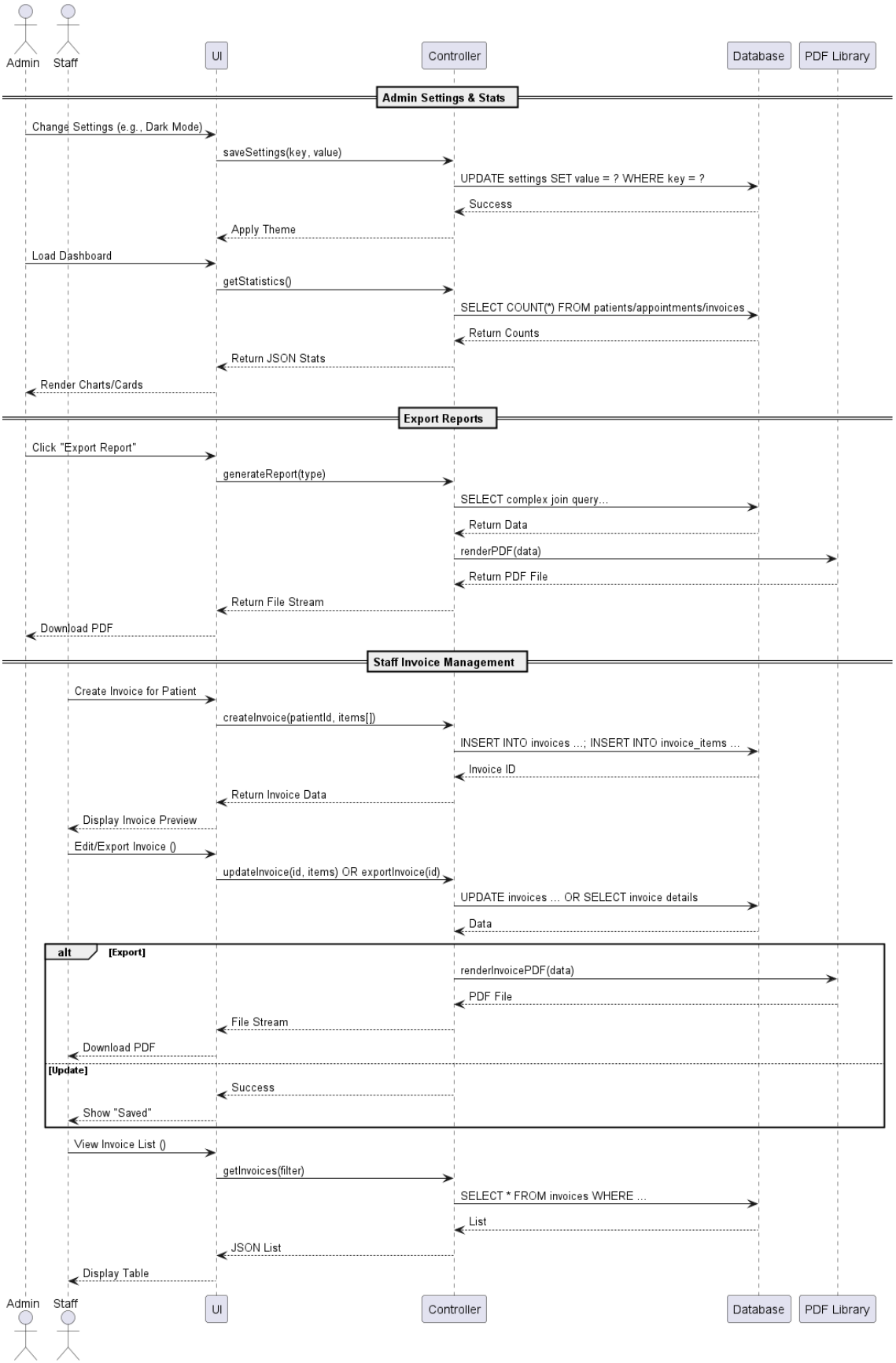


Figure (4.17) Sequence diagram finical and reporting operation

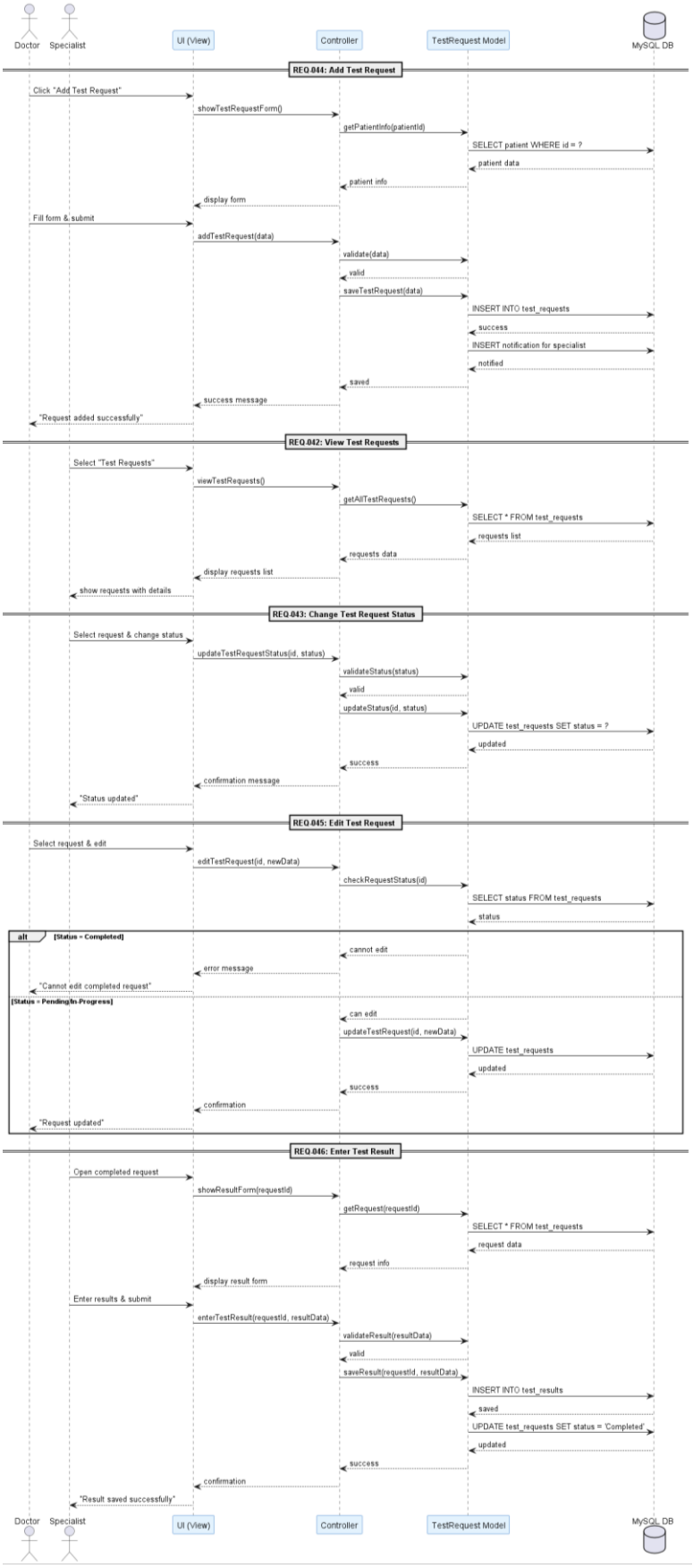


Figure (4.18) Sequence Diagram for Tests Management

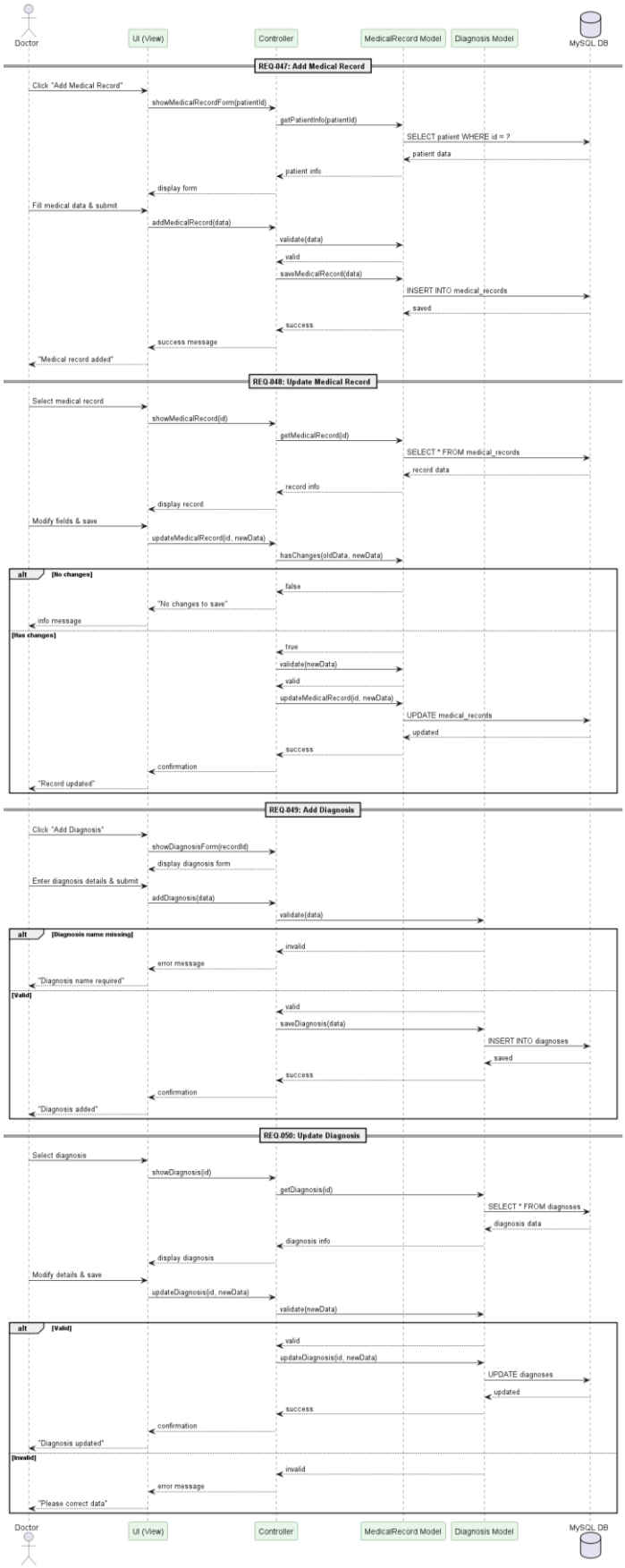


Figure (4.19) Sequence Diagram for Medical Records & Diagnosis Management

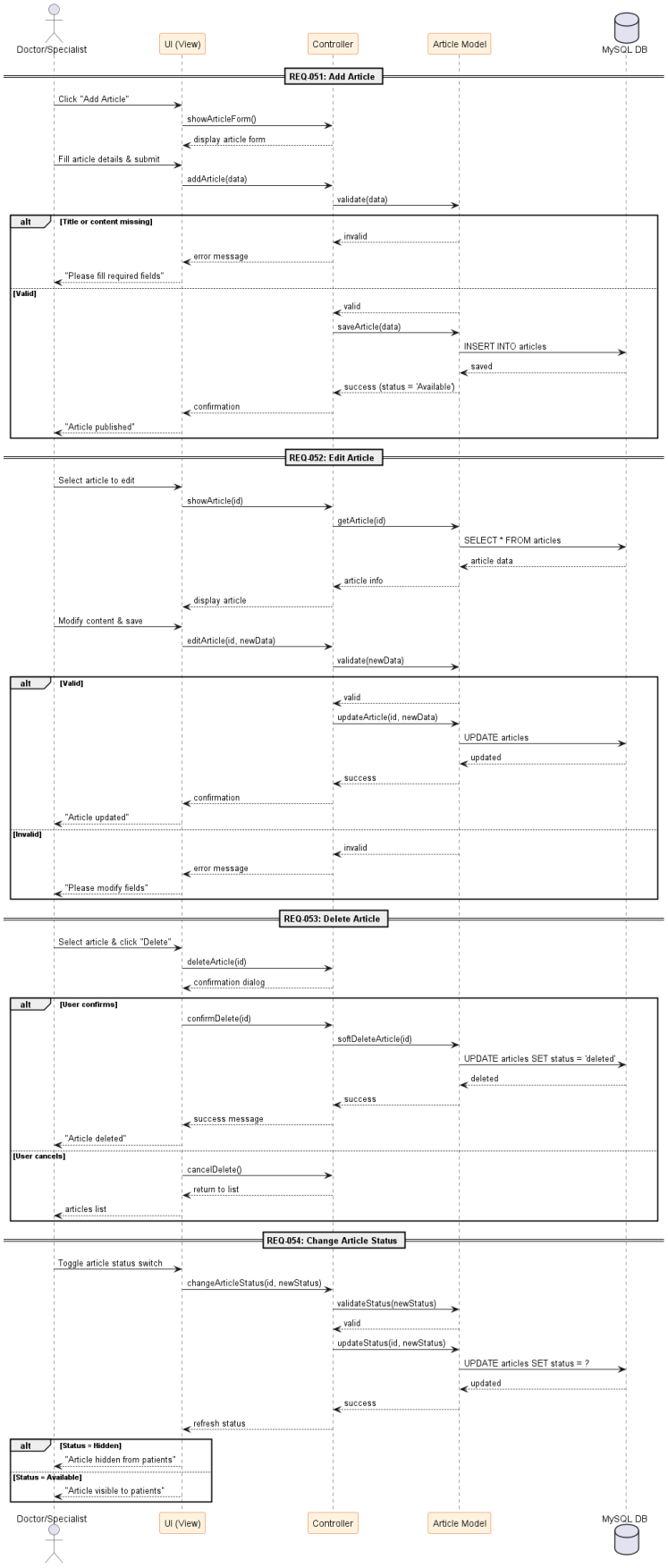


Figure (4.20) Sequence Diagram for Articles Management

Table (4.17) Sequence Diagram Key / Legend

Symbol	Description
Rectangle (Top)	Represents an Object or Lifeline (e.g., Doctor, UI, Controller, Database).
Dashed Vertical Line	The Lifeline, representing the existence of the object over time.
Solid Horizontal Arrow	Synchronous Message (Method call or query).
Dashed Horizontal Arrow	Reply Message (Return value or response).
Solid Arrow with Stick	Asynchronous Message (e.g., triggering a background process).
Box over Lifeline	Activation Bar (Execution occurrence specification).

4.5 Class Diagram:

The Class Diagram defines the static structural design of the system's object-oriented backend. It maps the database entities to PHP classes, detailing their attributes, methods, and relationships.

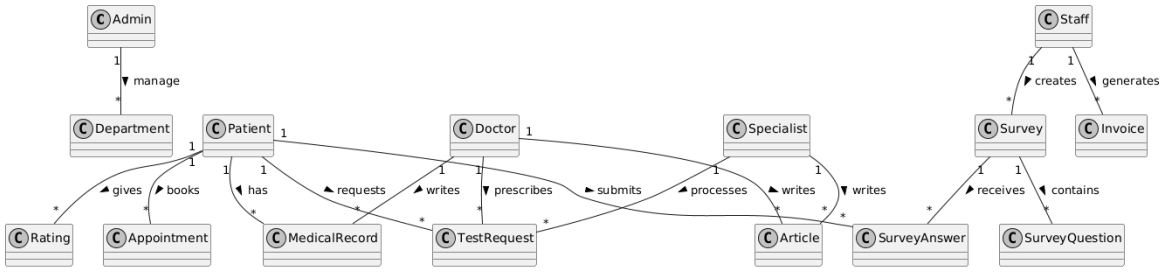


Figure (4.21) Class Diagram

Table (4.20) Class Diagram Key / Legend

Symbol	Description
Three-Tier Rectangle	Represents a Class (Name, Attributes, Methods).
Solid Line with Diamond	Aggregation/Composition (Lifecycle dependency between classes).
Solid Line with Arrow	Association / Dependency (One class uses another).
Hollow Triangle (Inheritance)	Generalization (e.g., Doctor inherits from User).
+, -, #	Visibility: Public, Private, Protected.

4.6 Component Diagram:

The Component Diagram illustrates the physical and logical components of the HCMS and their interfaces. It demonstrates how the modular PHP directories, external libraries (e.g., TCPDF for invoices), and the database interact to deliver the final system functionality.

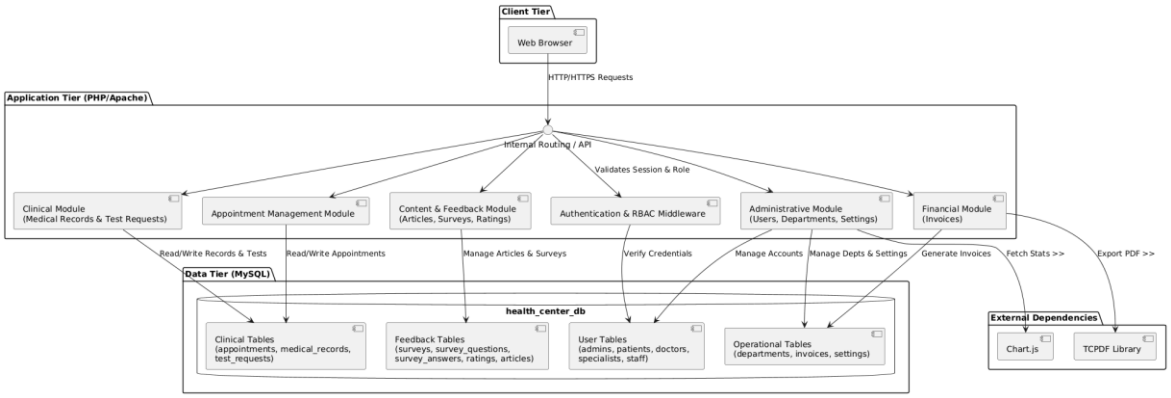


Figure (4.22) Component Diagram

Table (4.21) Component Diagram Key / Legend

Symbol	Description
Rectangle with Tabs	Represents a Software Component (e.g., Clinical Module, Auth Module).
Lollipop (Circle on stick)	Provided Interface (Services offered by the component).
Half-Circle	Required Interface (Services needed by the component).
Dashed Arrow	Dependency (Component A relies on Component B).
Cylinder	Database / Persistent Storage Component.

4.7 Entity-Relationship Diagram (ERD):

The ERD models the relational database schema, ensuring data integrity and eliminating redundancy.

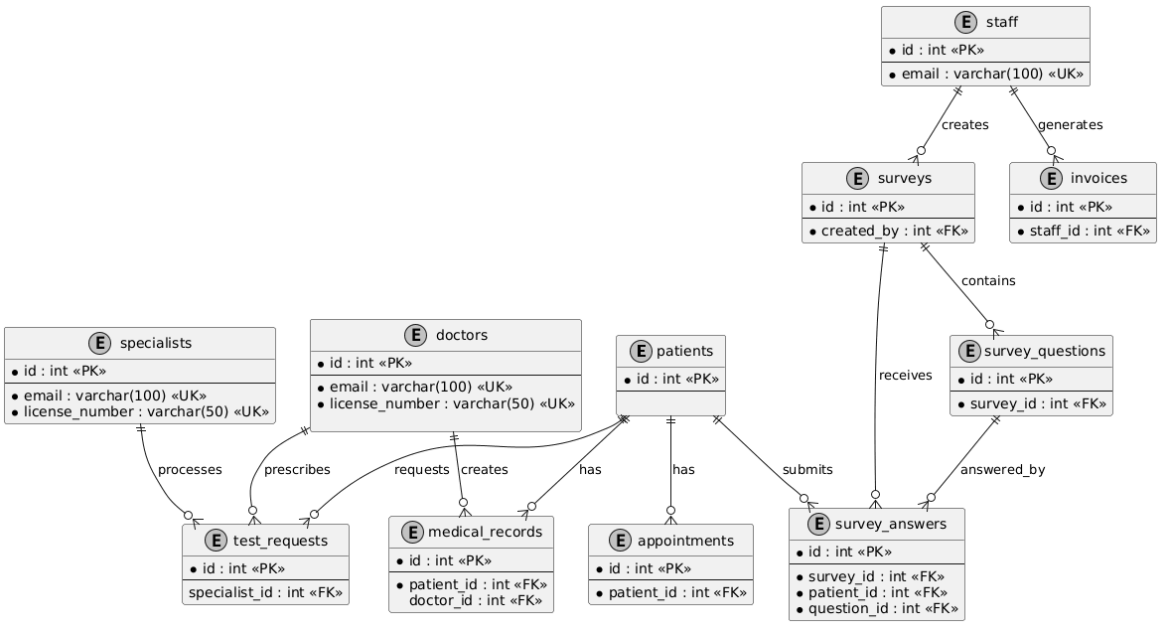


Figure (4.23) ERD

Table (4.22) ERD Key / Legend

Symbol	Description
Rectangle	Represents an Entity (Table in the database).
Oval	Represents an Attribute (Column).
Underlined Text	Primary Key (Unique identifier for the entity).
Dashed Oval	Derived Attribute (e.g., Total Invoice Amount).
Diamond	Relationship between entities (1:1, 1:N, M:N).
Line with Crow's Foot	Cardinality (e.g., One Patient has Many Medical Records).

4.8 System Sitemap:

The Sitemap provides a hierarchical tree view of the system's web pages, organized by user roles. It ensures logical navigation and reflects the strict RBAC routing implemented in the system.

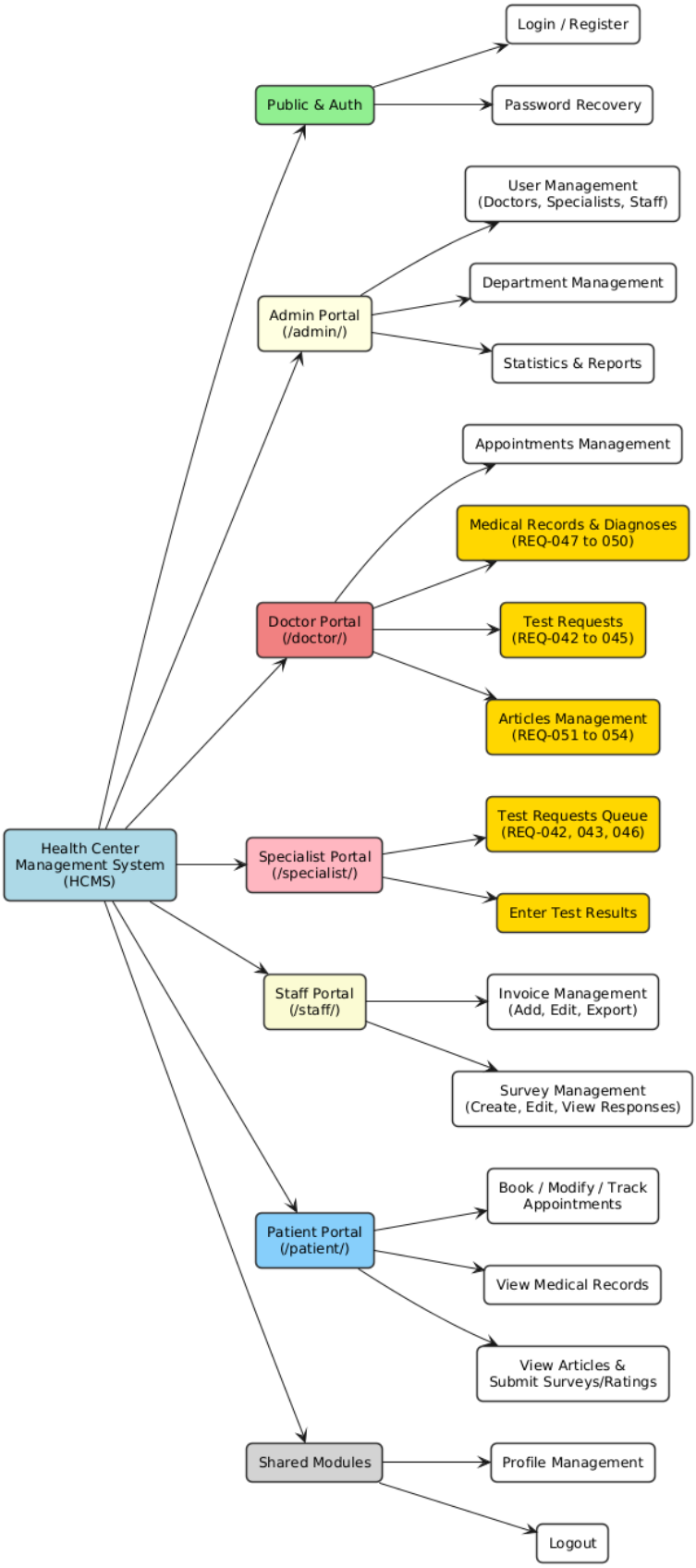


Figure (4.24) Sitemap

Table (4.23) Sitemap Key / Legend

Symbol	Description
Top Rectangle	**Root / Landing Page (Login / Home).**
Connected Rectangles	**Web Pages / Views grouped by User Role.**
Solid Lines	**Navigation hierarchy and routing paths.**
Color Coding	**Distinguishes between Public, Patient, Staff, Doctor, and Admin zones.**

4.9 Initial Test List (Unit Tests):

Before proceeding to integration testing, unit tests were designed to validate the core business logic and data access layers in isolation.

Table (4.24) Initial Unit Test List

Test ID	Module / Component	Test Description	Expected Outcome
UT-01	Authentication	Verify password_hash() and password_verify() logic.	Passwords are securely hashed and validated without plain-text storage.
UT-02	RBAC Middleware	Test auth_check.php with invalid session tokens.	System redirects to login page; access is denied.
UT-03	Data Access Layer	Execute PDO prepared statements with SQL injection payloads.	Database rejects malicious input; no syntax errors or data leaks.
UT-04	Clinical Logic	Validate TestRequest status transition (Pending -> In-Progress).	Status updates correctly in DB; invalid transitions are blocked.
UT-05	Content Logic	Verify Article visibility based on status ('Available' vs 'Hidden').	Hidden articles are excluded from the patient portal query results.

4.10 AI and Algorithms Design:

Artificial Intelligence: The current version of the HCMS does not utilize Machine Learning or AI predictive models. The architecture, however, is modularly designed to accommodate future AI integrations (e.g., predictive appointment no-shows or automated triage) via RESTful API endpoints.

Deterministic Algorithms: The system implements several critical rule-based algorithms:

1. **Appointment Conflict Resolution:** A temporal logic algorithm checks the appointments table to prevent double-booking of the same provider_id at overlapping time_slot intervals.
2. **Invoice Aggregation:** An iterative algorithm dynamically calculates the total_amount by summing service items and applying tax/discount rules before finalizing the database insert.
3. **Survey Analytics:** An aggregation algorithm computes weighted averages and response distributions for patient satisfaction surveys in real-time.

4.11 Requirements Traceability Matrix (RTM):

The RTM ensures that every requirement elicited in Chapter 3 is explicitly mapped to its corresponding design artifact, code module, and test case. This guarantees 100% coverage and prevents requirement slippage.

Table (4.25) Requirements Traceability Matrix

Req ID	Requirement Title	Design Artifact	Code Module	Test Case
REQ-001	Log in	Use Case, Sequence	/login.php, auth_check.php	TC-01, TC-02
REQ-019	View medical record	Use Case, ERD	/doctor/medical_records.php	TC-20
REQ-021	Book an appointment	Activity, Sequence	/patient/book_appointment.php	TC-21
REQ-042	**View test requests**	**Use Case, Activity**	**/specialist/view_tests.php**	**TC-42**
REQ-043	**Change test request status**	**Sequence, Class**	**/specialist/update_test_status.php**	**TC-43**
REQ-044	**Add a test request**	**Activity, ERD**	**/doctor/add_test_request.php**	**TC-44**
REQ-045	**Edit a test request**	**Sequence**	**/doctor/edit_test_request.php**	**TC-45**
REQ-046	**Enter test result**	**Activity, Class**	**/specialist/enter_result.php**	**TC-46**
REQ-047	**Add a medical record**	**ERD, Sequence**	**/doctor/add_medical_record.php**	**TC-47**
REQ-048	**Update medical record**	**Class**	**/doctor/update_record.php**	**TC-48**
REQ-049	**Add a diagnosis**	**ERD, Activity**	**/doctor/add_diagnosis.php**	**TC-49**
REQ-050	**Update a diagnosis**	**Sequence**	**/doctor/update_diagnosis.php**	**TC-50**
REQ-051	**Add an article**	**Use Case, ERD**	**/doctor/add_article.php**	**TC-51**
REQ-052	**Edit an article**	**Activity**	**/doctor/edit_article.php**	**TC-52**

Chapter 5
Implementation

5.1 Execution Environment:

The system was implemented and validated in a local development and demonstration environment based on the XAMPP distribution, which provides a cohesive integration of the Apache HTTP Server, the PHP runtime, and the MySQL relational database management system. This environment was selected on the basis of the technical and economic feasibility criteria established in Section 3.5, ensuring that the solution remains reproducible on standard institutional hardware without proprietary licensing costs.

Table (5.1) Implementation Environment and Toolchain

Layer	Tool / Technology	Version / Configuration	Purpose
Operating System	Windows 10/11 (64-bit)	Standard workstation	Local hosting and development platform
Web Server	Apache HTTP Server	2.4.x (via XAMPP)	HTTP request handling and PHP execution
Server-side Runtime	PHP	8.1.x	Business logic, session management, RBAC, data access
Database Engine	MySQL	8.0.x	Persistent relational storage
Database Administration	phpMyAdmin	Bundled with XAMPP	Schema management, data inspection, backup/export
Presentation Layer	HTML5, CSS3, JavaScript	Native + Bootstrap	Responsive interface with Arabic RTL support
PDF Generation	TCPDF	Latest stable release	Printable invoices and Arabic-compatible report export
Database Connector	PHP PDO	pdo_mysql driver	Parameterized queries and transaction support

Development Environment	Visual Studio Code	Current stable	Source editing and local debugging
Version Control	Git	Local repository + feature branches	Change tracking and requirement-linked commits
Client Browsers	Chrome / Edge	Current stable	Cross-browser validation and accessibility checks

The database connection is centralized in a single configuration file (includes/db.php), which instantiates a PDO connection using the DSN mysql:host=localhost;dbname=hcms;charset=utf8mb4. The utf8mb4 character set is enforced at the connection and schema level to guarantee correct storage and retrieval of Arabic content. The connection is configured with PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION and PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC, so that database failures are surfaced explicitly rather than silently ignored.

5.2 Software Project Structure:

The source code is organized according to a layered, convention-based structure that realizes the MVC concept described in Chapter 4. Although the project does not rely on a heavyweight framework, it enforces strict separation between presentation, business logic, and data access to preserve modularity and support future extension.

Table (5.2) Repository Directory Structure and Responsibilities

Directory / File	Responsibility
index.php	Application entry point; authenticates the session and routes the user to the role-specific dashboard
login.php / logout.php	Credential verification, session initialization, and secure session termination
includes/	Shared infrastructure: db.php (PDO connection), auth_check.php (RBAC middleware), header.php, footer.php, and functions.php
admin/	Account approval/deactivation, department lifecycle, system settings, statistics, and report export
doctor/	Appointment handling, medical records, diagnoses, test request creation, and article authoring
specialist/	Test request queue, status transitions, and laboratory result entry
staff/	Invoice creation and editing, survey management, and patient feedback review
patient/	Appointment booking and tracking, medical record viewing, article reading, ratings, and survey submission
assets/	Stylesheets (including RTL overrides), JavaScript modules, and uploaded media
uploads/	Secure server-side storage for article images and permitted attachments
out/	Generated artifacts such as PDF invoices and statistical summaries produced through TCPDF/FPDF
notes.txt	Developer log recording implementation decisions and outstanding issues
.qodo/	IDE/tooling metadata; excluded from the runtime deployment package

The routing convention maps each functional requirement to a dedicated, role-scoped script. For example, REQ-044 is realized by doctor/add_test_request.php, while its counterpart operation REQ-046 is realized by specialist/enter_result.php. This physical separation reinforces the authorization boundary between the prescribing role and the analyzing role.

Version control was maintained throughout implementation using descriptive, requirement-linked commit messages (e.g., Implement REQ-047: Medical record creation with transactional diagnosis insert) and feature branches following the convention feature/REQ-XXX. This practice preserved a direct audit trail between source history and the requirements baseline.